

YOLOv1 desde cero sobre COCO 2017: un estudio de reproducción con análisis forense de errores

Zakaria Abouhammadi¹

¹Universitat de València, Escola Tècnica Superior d'Enginyeria

Mayo de 2026

Resumen

Reproducimos el detector YOLOv1 de [Redmon et al. \(2016\)](#) desde cero en PyTorch, lo portamos del entorno VOC del artículo original al dataset COCO 2017 (ochenta clases) y lo entrenamos extremo a extremo con una red troncal ResNet18 preentrenada sobre ImageNet, un adaptador convolucional y la cabeza de detección densa de dos capas que propone el artículo. La contribución sustantiva de este trabajo es metodológica más que numérica. Las cuatro primeras ejecuciones de entrenamiento, que cubren dos redes troncales distintas y un barrido deliberadamente amplio de tasas de aprendizaje y umbrales de recorte de gradiente, produjeron exactamente $\text{mAP}@0,5 = 0$ sobre el conjunto de prueba a pesar de que las pérdidas de entrenamiento y validación descendían monotónicamente y, en aislamiento, parecían evidencia de un modelo que aprendía. Una auditoría del código identificó dos errores de implementación latentes en la función de pérdida y en el decodificador de coordenadas, cada uno de ellos suficiente por sí solo para fijar la métrica a cero sobre COCO, y a la vez invisibles tanto para la batería de tests existente (que ejercitaba únicamente la disposición de índices de VOC) como para la inspección de las curvas de entrenamiento. Ambos errores se corrigieron con un puñado de líneas y se verificaron mediante tests sintéticos de *round-trip* y de sensibilidad de la pérdida. Las dos ejecuciones posteriores a la corrección (V5 y V6) produjeron los primeros valores no nulos de detección, y el *checkpoint* de V6 alcanza $\text{mAP}@0,5 = 0,0415$ sobre el conjunto de prueba con una latencia de inferencia hacia adelante de $\sim 5,3$ ms por imagen de 448×448 en una única GPU portátil de generación Ada. No reclamamos competitividad con los detectores actuales: la distancia respecto al 63,4 que reporta el artículo original sobre VOC se explica por un dataset sustancialmente más difícil, un calendario de entrenamiento de la mitad de duración, una red troncal más pequeña y un pipeline de aumentado de datos mínimo; analizamos cada uno de estos factores por separado. La idea con la que queríamos dejar al lector es que una curva de pérdida convergente no constituye, por sí sola, evidencia de que el modelo está aprendiendo lo que la pérdida pretende medir; tests cuantitativos baratos de *round-trip* sobre cada transformación de coordenadas habrían detectado ambos errores antes de la primera época.

1. Introducción

La detección de objetos es uno de los problemas canónicos de la visión por computador. Dada una imagen de entrada, la tarea consiste en enumerar los objetos presentes, clasificar cada uno en una de un conjunto fijo de categorías y localizarlo mediante una caja delimitadora. La salida es, por tanto, de longitud variable y estructurada, lo que distingue la detección de la clasificación de imágenes (salida de longitud fija) y de la segmentación semántica (una predicción por píxel). La combinación de clasificación y localización ha sido un banco de pruebas productivo para el aprendizaje de representaciones desde el resurgimiento de las redes convolucionales profundas a principios de la década de 2010 (Krizhevsky et al., 2012; Russakovsky et al., 2015), y sigue siendo una capacidad fundamental detrás de aplicaciones que van desde la conducción autónoma a la imagen médica o la inspección industrial.

1.1. Contexto histórico

Dos familias algorítmicas han dominado la era del aprendizaje profundo en detección de objetos, con compensaciones marcadamente distintas entre precisión, velocidad y complejidad de implementación.

Detectores de dos etapas. La primera oleada, comenzando por R-CNN (Girshick et al., 2014), descompone el problema en dos etapas secuenciales: primero generar un conjunto de regiones candidatas donde podrían encontrarse objetos, y después clasificar cada candidata independientemente con una red convolucional. El mecanismo de propuesta de regiones de R-CNN (*selective search*) era externo y costoso; sus sucesores Fast R-CNN (Girshick, 2015) y Faster R-CNN (Ren et al., 2015) absorbieron progresivamente la fase de propuestas dentro de la propia red, intercambiando la cascada de propuestas discretas por una red de propuestas aprendida y por características convolucionales compartidas. Los detectores de dos etapas suelen alcanzar la mayor precisión de localización sobre los *benchmarks* estándar, a costa de un camino de inferencia que toca cada propuesta al menos dos veces.

Detectores de una sola etapa. La segunda oleada, de la cual YOLOv1 (Redmon et al., 2016) y SSD (Liu et al., 2016) son los representantes canónicos, reformula la detección como un problema de regresión densa única sobre una rejilla espacial fija. La red produce, en una sola pasada hacia adelante, un tensor completo cuyos canales codifican conjuntamente las puntuaciones de clase, los parámetros de las cajas y la confianza por caja en cada celda de la rejilla. No hay etapa de propuestas ni clasificador por región; el conjunto completo de detecciones predicho se lee de un único tensor de salida y se filtra mediante *non-maximum suppression*. Los detectores de una sola etapa suelen ser un orden de magnitud más rápidos que los de dos etapas y resultan idóneos para despliegue en tiempo real; el artículo original de YOLO subraya esta compensación ya en su propio título.

El éxito de YOLO desencadenó un esfuerzo investigador sostenido que produjo YOLOv2, YOLOv3 (Redmon and Farhadi, 2018) y una larga línea de descendientes de la comunidad, cada uno incorporando ideas (cajas ancla, predicción multi-escala, cabezas totalmente convolucionales, *focal loss*, aumentado por mosaicos) que reducen incrementalmente la diferencia de precisión

con los detectores de dos etapas conservando la estructura de inferencia en una sola pasada. Al momento de redactar este trabajo, la familia YOLO sigue siendo la opción práctica por defecto para detección en tiempo real sobre hardware estándar.

1.2. Por qué estudiar YOLOv1 en particular

Este trabajo se centra en el artículo original de YOLOv1 por tres razones.

Primero, es el detector más simple de la familia de una sola etapa. La red es una pila de convoluciones seguida de dos capas densas; la salida es un único tensor; la pérdida es una sola expresión de suma de errores cuadrados con cinco términos claramente delimitados. Todo lo que añaden los descendientes (anclas, FPN, salidas multi-escala, *focal loss*) está ausente. Una reproducción exitosa ejercita por tanto cada uno de los conceptos que los trabajos posteriores asumen que el lector ya entiende, sin introducir todavía las optimizaciones que los oscurecen.

Segundo, el artículo es inusualmente claro respecto a sus propias limitaciones. La sección 4.4 de [Redmon et al. \(2016\)](#) enumera modos de fallo (agrupaciones de objetos pequeños, relaciones de aspecto inusuales, localización espacial gruesa) que son inherentes a la formulación de YOLOv1 y que siguen siendo útiles de estudiar en su forma original, porque motivan cada cambio arquitectónico de las versiones posteriores de la familia.

Tercero, el esfuerzo de implementación es lo suficientemente reducido para ser auditable extremo a extremo por una sola persona, condición previa para el tipo de narrativa de *análisis forense de errores* que este trabajo enfatiza en la Sección 4.6. La auditoría identifica dos errores de implementación que la batería de tests existente no detecta, y los utiliza para argumentar a favor de un estilo concreto de tests cuantitativos del que los proyectos de aprendizaje profundo en producción se benefician, pero tienden a no escribir.

1.3. Objetivos de este trabajo

Los objetivos son deliberadamente reproductivos más que originales. Son:

1. *Reproducir el pipeline de YOLOv1 extremo a extremo en PyTorch.* El sistema completo — clase de dataset, modelo, pérdida, bucle de entrenamiento distribuido con precisión mixta, evaluador de mAP y script de inferencia en tiempo real — está escrito desde cero y vive en el repositorio asociado a este trabajo. La implementación sigue a [Redmon et al. \(2016\)](#) en cada punto donde el artículo es explícito y queda documentada en los puntos donde no lo es.
2. *Portar el entorno VOC del artículo ($C = 20$) a COCO 2017 ($C = 80$).* Los cambios arquitectónicos son mínimos (un ensanchamiento de la capa de salida y un ajuste en la disposición del tensor por celda), pero el régimen de entrenamiento es sustancialmente más duro, como analiza con detalle la Sección 6.1.
3. *Documentar la realidad ingenieril de entrenar un detector desde cero.* Esto incluye las cuatro iteraciones de hiperparámetros que no resolvieron el problema (Secciones 4.2–4.5), la auditoría de código que sí lo hizo (Sección 4.6), las ejecuciones posteriores a la corrección

que produjeron la primera métrica no nula (Secciones 4.7–4.8) y el desglose por clase que motiva el programa de trabajo futuro de la Sección 6.4.

No reclamamos competitividad con los detectores actuales, ni presentamos ninguna novedad metodológica. La contribución de este trabajo es la auditabilidad de un pipeline completo de YOLOv1 dirigido a COCO, junto con la documentación de dos modos de fallo que son fáciles de introducir, difíciles de detectar desde las curvas de entrenamiento y que merece la pena señalar a cualquiera que reproduzca un detector por primera vez.

1.4. Estructura del documento

La Sección 2 formaliza el problema y la pérdida de YOLOv1, resumiendo las partes de [Redmon et al. \(2016\)](#) en las que se apoyan las secciones siguientes. La Sección 3.1 describe el pipeline de preparación del dataset; la Sección 3.2 introduce la arquitectura del modelo como un único diagrama TikZ; las Secciones 3.3 a 3.5 cubren la implementación de la pérdida, la infraestructura de entrenamiento y el pipeline de evaluación. La Sección 4 es la narrativa sustantiva del trabajo: seis ejecuciones de entrenamiento, el fracaso de las cuatro primeras, la auditoría que explicó los fracasos y las dos que produjeron la primera métrica de detección no nula. La Sección 5 reporta las cifras titulares, el desglose por clase, la latencia de inferencia y una comparación lado a lado con el artículo original. La Sección 6.1 discute las limitaciones y la continuación natural del trabajo, y las conclusiones reformulan el hallazgo central. Dos apéndices contienen la tabla completa de AP por clase y un listado lado a lado de la función de pérdida antes y después de la corrección de índices.

2. Marco teórico

Esta sección formaliza el problema de detección de objetos, presenta la formulación en rejilla de YOLOv1 de [Redmon et al. \(2016\)](#), deriva la función de pérdida de cinco términos utilizada a lo largo de este trabajo y define las métricas empleadas en la evaluación. La exposición sigue de cerca al artículo; anotamos los puntos en los que nuestro entorno sobre COCO se aparta del entorno VOC del artículo.

2.1. La detección de objetos como problema de predicción estructurada

Un detector de objetos de una sola imagen es una función

$$f_{\theta} : \mathbb{R}^{3 \times H \times W} \longrightarrow \{ (c_i, b_i, s_i) \}_{i=1}^{n(x)}$$

que toma una imagen RGB y produce un conjunto de detecciones de longitud variable, cada una compuesta por una etiqueta de clase $c_i \in \{1, \dots, C\}$, una caja delimitadora $b_i \in [0, 1]^4$ en alguna parametrización acordada y una puntuación de confianza $s_i \in [0, 1]$. El número de detecciones $n(x)$ es a su vez función de la imagen, lo que distingue la detección de la clasificación (salida de longitud fija) y de la segmentación semántica (una predicción por píxel).

Dos familias algorítmicas han abordado históricamente este problema:

Detectores de dos etapas. R-CNN (Girshick et al., 2014) y sus sucesores Fast R-CNN (Girshick, 2015) y Faster R-CNN (Ren et al., 2015) generan primero un conjunto de candidatas regiones de interés (ya sea mediante propuestas clásicas como *selective search* o mediante una red de propuestas aprendida) y a continuación clasifican cada región de forma independiente. El desacoplamiento entre localización y clasificación produce alta precisión pero requiere múltiples pasadas hacia adelante por imagen.

Detectores de una sola etapa. YOLOv1 (Redmon et al., 2016) y SSD (Liu et al., 2016) predicen clase y caja conjuntamente a partir de una sola pasada hacia adelante de una red convolucional. La precisión de localización es típicamente menor que en los detectores de dos etapas, pero la inferencia es lo suficientemente rápida para uso en tiempo real.

YOLOv1 es el detector de una sola etapa canónico y el sujeto de este trabajo. Resumimos su idea central en la siguiente subsección.

2.2. Formulación en rejilla de YOLO

YOLOv1 reformula la detección como un problema de regresión estructurada sobre una rejilla fija. La imagen de entrada se divide en una matriz de $S \times S$ celdas. Cada celda es responsable de detectar los objetos cuyo centro cae dentro de ella. Para cada celda la red produce:

- B cajas delimitadoras candidatas, cada una parametrizada por $(x, y, w, h, \text{conf})$, donde (x, y) es el centro de la caja y conf es la IoU predicha entre esta caja y el objeto de referencia del que la celda es responsable.
- Un único conjunto de C probabilidades de clase, compartido entre las B cajas (un conjunto de puntuaciones de clase por celda, no por caja).

La salida por celda tiene por tanto $C + 5B$ canales, y el tensor completo de predicción tiene forma $(S, S, C + 5B)$. Para el entorno VOC del artículo original ($S = 7, B = 2, C = 20$) esto da $S^2(C + 5B) = 1470$ valores de salida por imagen. Para el entorno COCO utilizado en este trabajo ($C = 80$), crece a 4410.

Parametrización de coordenadas. Las coordenadas del centro (x, y) de una caja predicha se expresan *relativas a la celda* que contiene el centro de referencia, por lo que viven en $[0, 1]$ con $(0, 0)$ en la esquina superior-izquierda de la celda y $(1, 1)$ en la inferior-derecha. La anchura y la altura (w, h) se expresan *relativas a la imagen completa*, también en $[0, 1]$. Esta asimetría es deliberada: un objeto puede abarcar varias celdas, pero su centro cae en una y solo una. Coordenadas relativas a la celda forzarían a w, h a superar 1 para cualquier objeto no diminuto, lo que interactuaría mal con la transformación $\sqrt{\cdot}$ de la pérdida. Esta es la misma asimetría que causó el Error 2 de la Sección 4.6.2.

Disposición del tensor por celda. Los $C + 5B$ canales en cada celda se ordenan como

$$\underbrace{[p_1, \dots, p_C]}_{\text{probs. de clase}} \underbrace{[\text{conf}_1, x_1, y_1, w_1, h_1]}_{\text{caja 1}} \underbrace{[\text{conf}_2, x_2, y_2, w_2, h_2]}_{\text{caja 2}} \dots$$

con los canales de la segunda caja inmediatamente a continuación de los de la primera, y así sucesivamente para $B > 2$. El tensor de referencia usa la misma disposición pero solo rellena el primer hueco de caja: hay una caja de referencia por celda, no B . El segundo hueco está estructuralmente presente en el *target* pero queda a cero, y únicamente lo lee el lado de las predicciones al calcular la pérdida. Esta es la disposición concreta que se indizó mal en V1–V4 (cf. Sección 4.6.1).

La restricción de un objeto por celda. En tiempo de entrenamiento, si dos centros de referencia caen en la misma celda, solo uno de ellos se conserva; el otro se descarta. Por construcción, el detector no puede recuperar ambos. Es una limitación dura de la formulación de YOLOv1 y uno de los modos de fallo principales observados en la Sección 5.2, donde las clases que coocurren con frecuencia en agrupaciones densas (tenedores, cuchillos y cucharas sobre mesas; varias pelotas en fotografías de primer plano) registran un AP particularmente bajo.

2.3. La pérdida de cinco términos

El objetivo de entrenamiento es una suma de errores cuadrados sobre cinco términos que implementa lo que [Redmon et al. \(2016\)](#) describen como una pérdida multitarea que equilibra localización, confianza y clasificación. La expresión completa es

$$\begin{aligned}
\mathcal{L} = & \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{K}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] \\
& + \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{K}_{ij}^{\text{obj}} \left[(\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2 \right] \\
& + \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{K}_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 \\
& + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{K}_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2 \\
& + \sum_{i=0}^{S^2} \mathbb{K}_i^{\text{obj}} \sum_{c \in \text{clases}} (p_i(c) - \hat{p}_i(c))^2
\end{aligned} \tag{1}$$

donde las cantidades con sombrero son predicciones y las que no lo llevan son referencia. Las dos funciones indicadoras se definen como

$$\mathbb{K}_{ij}^{\text{obj}} = \begin{cases} 1 & \text{si la caja } j \text{ de la celda } i \text{ es responsable de un objeto,} \\ 0 & \text{en caso contrario,} \end{cases} \quad \mathbb{K}_{ij}^{\text{noobj}} = 1 - \mathbb{K}_{ij}^{\text{obj}},$$

siendo $\mathbb{K}_i^{\text{obj}}$ el OR a nivel de celda sobre las B cajas (vale 1 si y solo si la celda i contiene un centro de referencia). Los cinco términos se leen de las filas de la Ecuación (1); una versión anotada en forma de diagrama aparece en la Figura 1.

loss function explained

$$\begin{aligned}
 & \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{I}_{i,j}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] \quad \text{Squared error for box coordinates: the midpoint x-value minus the predicted x, squared, plus the midpoint y-value minus the predicted y, squared} \\
 & \quad \text{It is a scaling factor used to balance the importance of localization accuracy relative to the other terms in the loss function} \quad \text{Identity function that is going to be 1 or 0: it is going to be 1 if there is a box in cell (where goes from 0 to S^2) and if bounding box j is responsible for that prediction} \quad \text{Highest IOU} \\
 & + \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{I}_{i,j}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right] \quad \text{The square root is used for the width (w) and height (h) to ensure that small deviations in small boxes matter more than the same deviations in large boxes. Since the derivative of the square root function decreases as the value increases, it naturally penalizes errors in small objects more heavily, providing better scale invariance during training.} \\
 & + \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{I}_{i,j}^{\text{obj}} \left(C_i - \hat{C}_i \right)^2 \quad \text{measures the error between the predicted confidence score and the actual confidence} \\
 & \quad \text{In this stage, the network does not care about the object's class it only learns to estimate the probability that an object actually exists within the predicted bounding box} \\
 & + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{I}_{i,j}^{\text{noobj}} \left(C_i - \hat{C}_i \right)^2 \quad \text{The identity function (noobj) is only activated when there is no object in cell. Since the ground truth is 0 in these cases, this term penalizes the network by forcing the predicted confidence toward zero. This ensures the model learns to ignore the background and reduces the appearance of false positives.} \\
 & + \sum_{i=0}^{S^2} \mathbb{I}_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2 \quad \text{the target probability } p_i(c) \text{ is 1 for the correct class and 0 for all others. This forces the network to maximize the probability of the actual class while simultaneously pushing the probabilities of all incorrect classes toward zero.}
 \end{aligned}$$

Figura 1: Diagrama anotado de la pérdida de YOLOv1. Los cinco términos se muestran con la notación del artículo original, con anotaciones manuscritas que describen el propósito de cada sumando. La figura es una anotación propia sobre el diagrama de pérdida de Redmon et al. (2016).

Lectura término a término.

1. *Término del centro.* Penaliza el error cuadrado en (x, y) en las celdas con objetos, sumado únicamente sobre la caja responsable. Ponderado por $\lambda_{\text{coord}} = 5$ para que la localización reciba más señal de gradiente que la clasificación, que tiene muchos más sumandos contribuyentes.
2. *Término de tamaño.* Penaliza el error cuadrado en $\sqrt{w}$ y $\sqrt{h}$ en lugar de directamente en w y h . La raíz cuadrada comprime geométricamente el rango dinámico de los tamaños de caja: un error de 0,1 sobre una caja de anchura 0,1 es mucho más importante que el mismo error absoluto sobre una caja de anchura 0,9. Sin la raíz cuadrada, las cajas grandes dominarían el gradiente y los objetos pequeños nunca se localizarían correctamente.
3. *Término de confianza de objeto.* Para cada celda que contiene un objeto, la confianza del predictor responsable se atrae hacia la IoU entre su caja y la caja de referencia. La asignación de responsabilidad usa la mayor IoU entre las B cajas candidatas por celda (ver más abajo).
4. *Término de confianza sin objeto.* Para cada caja en cada celda que no contiene un objeto, la confianza se atrae hacia cero. Este término aporta con mucho el mayor número de sumandos; sin la ponderación a la baja por $\lambda_{\text{noobj}} = 0,5$, dominaría la señal por objeto y el modelo convergería a predecir confianza cero en todas partes.
5. *Término de clase.* Atrae la distribución predicha de clases en cada celda con objeto hacia la distribución de referencia *one-hot*. Este término se calcula una vez por celda (no por caja), lo que equivale a decir que todas las B cajas de una celda comparten la misma predicción de clase.

Responsabilidad. El artículo original asigna la responsabilidad de un objeto a la caja predicha con mayor IoU contra la caja de referencia. En tiempo de entrenamiento, para cada celda que contiene un objeto, las dos cajas predichas se evalúan contra la única caja de referencia; la caja con mayor IoU se marca como responsable, y solo su tupla $(x, y, w, h, \text{conf})$ contribuye a los términos de centro, tamaño y confianza de objeto. La caja perdedora contribuye solo a través del término de confianza sin objeto. La intuición es que las dos cajas por celda no son redundantes: a lo largo del entrenamiento se especializan en distintas relaciones de aspecto (una alargada, otra ancha, por ejemplo), y la asignación de responsabilidad actúa como selector suave.

La trampa de $\sqrt{\cdot}$. La salida de la red no está restringida, así que un w o h predicho puede ser negativo al inicio del entrenamiento. La raíz cuadrada de un número negativo no está definida para tensores reales; si la pérdida se implementa de manera ingenua como `torch.sqrt(w)`, la primera predicción negativa produce un NaN que se propaga por el resto del *batch*. Una implementación robusta utiliza $\text{sign}(x) \cdot \sqrt{|x| + \varepsilon}$, preservando el signo para que el gradiente del paso siguiente apunte de vuelta hacia el semieje positivo. Esta es la forma utilizada en `src/loss.py` y se discute en la Sección 3.3.

2.4. Intersection over Union

El cociente *Intersection-over-Union* (IoU, también llamado índice de Jaccard) es la medida geométrica fundamental utilizada a lo largo de la pérdida y de la métrica de evaluación. Para dos cajas delimitadoras A y B alineadas con los ejes, expresadas en el mismo sistema de coordenadas, se define como

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|},$$

donde $|\cdot|$ denota el área de la caja. El cociente toma valores en $[0, 1]$, con 0 para cajas disjuntas y 1 para cajas idénticas. En la pérdida, la IoU se calcula entre cada una de las B cajas predichas y la única caja de referencia por celda para asignar responsabilidad. En la evaluación, la IoU se calcula entre toda predicción y toda caja de referencia de la misma clase para decidir si la predicción es un verdadero o falso positivo.

Nota de implementación. Para dos cajas dadas en formato de centro (x_c, y_c, w, h) , el rectángulo de intersección tiene esquinas

$$x_1 = \max(x_c^A - w^A/2, x_c^B - w^B/2), \quad x_2 = \min(x_c^A + w^A/2, x_c^B + w^B/2),$$

y análogamente para y . Su área es $\max(0, x_2 - x_1) \cdot \max(0, y_2 - y_1)$; el *clamp* en cero gestiona el caso sin solapamiento, que de otro modo produciría un “área” negativa. El denominador añade un pequeño $\varepsilon = 10^{-6}$ para evitar la división por cero cuando ambas cajas tienen área nula.

2.5. Mean Average Precision

El *Mean Average Precision* a un umbral de IoU τ ($\text{mAP@}\tau$) es la métrica de detección estándar introducida por el reto Pascal VOC (Everingham et al., 2010) y adoptada por COCO (Lin et al.,

2014). Captura tanto la calidad de localización de objetos (a través del umbral de IoU) como la calidad del orden por confianza (a través de la integración de precisión–recall).

Average Precision por clase. Fijada una clase c y un conjunto de prueba con N_c instancias de referencia de esa clase a lo largo de todas las imágenes, el modelo produce una lista de predicciones $\{(s_k, \text{caja}_k)\}_{k=1}^{M_c}$ para la clase c , siendo s_k la confianza predicha. Se ordena la lista por confianza descendente. Se recorre en ese orden, manteniendo un recuento de verdaderos positivos (TP) y falsos positivos (FP). Una predicción es verdadero positivo si tiene $\text{IoU} \geq \tau$ frente a una caja de referencia *que no haya sido reclamada* por una predicción anterior (de mayor confianza); en caso contrario es un falso positivo. Una vez reclamada una caja de referencia, las predicciones posteriores que se solapan con ella cuentan como duplicados y se registran como falsos positivos.

Tras el paso k del recorrido, la precisión y el *recall* acumulados son

$$P_k = \frac{\text{TP}_k}{\text{TP}_k + \text{FP}_k}, \quad R_k = \frac{\text{TP}_k}{N_c}.$$

El Average Precision para la clase c es el área bajo la curva de precisión–recall, evaluada por integración trapezoidal con un anclaje adicional en $(R, P) = (0, 1)$ para que la integración esté bien definida en el extremo izquierdo:

$$\text{AP}_c = \int_0^1 P_c(R) dR \approx \sum_{k=1}^{M_c} \frac{P_{k-1} + P_k}{2} (R_k - R_{k-1}).$$

El mean Average Precision es entonces la media de los AP por clase sobre las C clases:

$$\text{mAP@}\tau = \frac{1}{C} \sum_{c=1}^C \text{AP}_c(\tau).$$

El reto VOC original fija $\tau = 0,5$; COCO además reporta el mAP promediado sobre $\tau \in \{0,5, 0,55, \dots, 0,95\}$, denotado $\text{mAP@[0,5:0,05:0,95]}$. En este trabajo reportamos únicamente mAP@0.5 , tanto porque es la métrica titular de [Redmon et al. \(2016\)](#) como porque, en este nivel de rendimiento absoluto, el umbral de 0.5 separa ya las clases de forma significativa.

El registro de cajas reclamadas. El detalle que distingue al mAP de un cálculo ingenuo de precisión/*recall* es la regla por la que cada caja de referencia puede ser reclamada por una sola predicción. Dos predicciones que ambas solapen con la misma caja de referencia con $\text{IoU} \geq \tau$ cuentan como un TP y un FP en lugar de como dos TPs; el FP es el duplicado. Este es el mecanismo que penaliza a un modelo que se cubre las espaldas emitiendo múltiples predicciones de alta confianza sobre el mismo objeto. En nuestra implementación, el estado reclamado se mantiene como un vector booleano por imagen y por clase, indizado por la posición de la caja de referencia en la lista de referencias por imagen.

Saltar las clases vacías. Una clase que tiene cero instancias de referencia en el conjunto evaluado tiene un AP indefinido (división por cero en el cálculo del *recall*). La convención

utilizada en este trabajo, y en la mayoría de las implementaciones públicas de mAP, es descartar dichas clases del promedio en lugar de contarlas como $AP = 0$. Sobre COCO esto nunca se activa porque todas las clases están representadas en el conjunto de prueba, pero la regla importa cuando este mismo código se reutiliza sobre *benchmarks* más pequeños.

2.6. Redes troncales y aprendizaje por transferencia

Las ecuaciones de la cabeza de detección expuestas arriba son agnósticas respecto a qué red troncal convolucional produce el mapa de características que alimenta a las capas densas. Dos opciones son relevantes para este trabajo.

Darknet24 es la red convolucional propia de 24 capas propuesta en Redmon et al. (2016).

Alterna convoluciones 3×3 y 1×1 , usa activaciones LeakyReLU y los autores del artículo la preentrenan sobre ImageNet antes del paso de *fine-tuning* para detección. Reproducimos esta red troncal para V1–V3 y la entrenamos desde cero (es decir, sin el paso de preentrenamiento sobre ImageNet), con las consecuencias descritas en las Secciones 4.2 a 4.4.

ResNet18 es el miembro más pequeño de la familia de redes residuales de He et al. (2016).

Las redes residuales introducen conexiones de atajo de identidad que permiten que los gradientes fluyan a través de pilas profundas de capas convolucionales sin desvanecerse, y se han convertido en la red troncal de clasificación sobre ImageNet por defecto en buena parte del campo. El paquete `torchvision` de PyTorch distribuye ResNet18 con un *checkpoint* preentrenado sobre ImageNet, que usamos directamente para inicializar V4–V6.

Aprendizaje por transferencia. La razón por la que el preentrenamiento sobre ImageNet importa es que las características aprendidas para clasificar imágenes naturales a resolución 224×224 son también útiles para localizar esos mismos objetos a resoluciones superiores. Una red convolucional inicializada al azar y de profundidad comparable, enfrentada al problema de arranque en frío de aprender simultáneamente aristas de bajo nivel, texturas de nivel medio y partes de objeto de alto nivel mientras aprende también la cabeza específica de detección, requiere típicamente sustancialmente más datos de entrenamiento y más pasos que la misma red inicializada desde un *checkpoint* de ImageNet. Las ejecuciones V1–V3 entrenadas desde cero sobre COCO ilustran el coste práctico de saltarse este paso. La ejecución V4, que cambió a una ResNet18 preentrenada, fue la primera que convergió a una pérdida de validación baja dentro del presupuesto asignado, aunque como discute la Sección 4.5, una pérdida de validación baja no resultó ser suficiente dados los errores que V4 todavía heredaba.

Esto concluye los preliminares teóricos. La siguiente sección traduce cada una de estas definiciones formales a código PyTorch ejecutable.

3. Implementación

Esta sección describe las decisiones de ingeniería que convierten la formulación de YOLOv1 de la Sección 2 en un sistema PyTorch ejecutable. Cubrimos, en orden, el pipeline de preparación de datos, la arquitectura del modelo, la implementación de la pérdida, la infraestructura de

entrenamiento y el pipeline de evaluación. Los errores concretos que afectaron a V1–V4 no se discuten aquí; pertenecen a la narrativa de la Sección 4.6.

3.1. Preparación del dataset

El pipeline de entrenamiento asume un único dataset consolidado, limpio y pre-particionado bajo `data/coco/`. Construirlo a partir de un archivo COCO 2017 recién descargado es un proceso en tres pasos que ejecutamos una vez, fuera de línea, antes de la primera ejecución de entrenamiento.

Paso 1 — consolidación. El archivo COCO 2017 se distribuye en dos carpetas de imágenes disjuntas (`train2017/` con 118 287 imágenes y `val2017/` con 5 000) y dos ficheros de anotaciones correspondientes. `scripts/consolidate_dataset.py` los une en un único directorio plano `images/` con 123 287 imágenes y un único `instances_all.json`. Los nombres de fichero de COCO son globalmente únicos, por lo que el traslado se realiza sin colisiones.

Paso 2 — limpieza. El fichero de anotaciones distribuido ocupa aproximadamente 550 MB; la mayor parte de esa masa son polígonos de segmentación, descripciones y metadatos de licencia que no usamos. `scripts/clean_dataset.py` reduce cada registro al mínimo necesario para la clase de dataset: (`id`, `file_name`, `width`, `height`) por imagen, (`image_id`, `bbox`, `category_id`) por anotación, y (`id`, `name`) por categoría. En la misma pasada elimina las 10 498 anotaciones marcadas `iscrowd=1`, pensadas como regiones a ignorar y que confundirían a un detector de una sola caja por celda. El fichero limpio es aproximadamente siete veces más pequeño y carga en menos de tres segundos sobre el NVMe del nodo H200.

Paso 3 — partición determinista. `scripts/prepare_splits.py` realiza un único reparto aleatorio 90%/5%/5% con semilla 42, produciendo tres ficheros en formato COCO: `instances_train.json` (110 957 imágenes), `instances_val.json` (6 165 imágenes) e `instances_test.json` (6 165 imágenes). La semilla queda fijada para que el conjunto de prueba quede genuinamente reservado a lo largo de las seis ejecuciones de entrenamiento; cualquier reordenación entre V1 y V6 habría imposibilitado comparar sus métricas de detección en igualdad de condiciones.

Conversión de anotaciones. Dentro de la clase de dataset (`src/dataset.py`), cada caja delimitadora de COCO ($x_{tl}, y_{tl}, w_{px}, h_{px}$) en coordenadas *en píxeles absolutos* se convierte al formato de rejilla de YOLO utilizado en el resto del pipeline:

$$x_{\text{norm}} = \frac{x_{tl} + w_{px}/2}{W_{\text{img}}}, \quad y_{\text{norm}} = \frac{y_{tl} + h_{px}/2}{H_{\text{img}}}, \quad (2)$$

$$w_{\text{norm}} = \frac{w_{px}}{W_{\text{img}}}, \quad h_{\text{norm}} = \frac{h_{px}}{H_{\text{img}}}. \quad (3)$$

A continuación, la caja delimitadora se asigna a la única celda de la rejilla $(i, j) = (\lfloor S y_{\text{norm}} \rfloor, \lfloor S x_{\text{norm}} \rfloor)$ cuyas coordenadas del centro dentro de esa celda son $x_{\text{cell}} = S x_{\text{norm}} - j$ y $y_{\text{cell}} = S y_{\text{norm}} - i$. La anchura y la altura se mantienen relativas a la imagen; esta asimetría entre $(x_{\text{cell}}, y_{\text{cell}})$ y $(w_{\text{norm}}, h_{\text{norm}})$ es el origen del Error 2 discutido en la Sección 4.6.2.

Un objeto por celda. Cuando dos centros de cajas de referencia caen en la misma celda, la clase de dataset conserva únicamente el primero encontrado en el orden de iteración. Esto implementa la restricción arquitectónica que enuncia Redmon et al. (2016): “cada celda de la rejilla predice un único conjunto de probabilidades de clase, con independencia del número de cajas B ”. No intentamos detectar ni reportar estas colisiones durante el entrenamiento; sobre COCO son infrecuentes para $S = 7$ en relación al total de anotaciones, pero responsables de una fracción no despreciable de las detecciones perdidas discutidas en la Sección 5.2.

Transformación de entrada. Como la red troncal en V4–V6 se inicializa desde el *checkpoint* preentrenado sobre ImageNet de torchvision, el tensor de entrada debe seguir la convención de normalización de ImageNet. El dataset aplica $\mu_{\text{RGB}} = (0,485, 0,456, 0,406)$ y $\sigma_{\text{RGB}} = (0,229, 0,224, 0,225)$ tras ToTensor (Krizhevsky et al., 2012; Russakovsky et al., 2015). El código de visualización en `src/visualization.py` deshace esta normalización antes de renderizar con PIL; no hacerlo produce tensores con la forma correcta pero distribuciones de color visualmente incoherentes, lo que ocultaría problemas de localización durante la revisión cualitativa.

3.2. Arquitectura del modelo

La arquitectura utilizada en V4, V5 y V6 es un pipeline de tres etapas: una red troncal ResNet18 preentrenada, un adaptador aprendido que conecta la resolución espacial de la red troncal con la cabeza de detección, y una cabeza de detección densa cuya capa final corresponde a la salida de rejilla de YOLO. El flujo completo de datos se resume en la Figura 2.

Elección de la red troncal. El YOLOv1 original (Redmon et al., 2016) utiliza una red troncal Darknet propia de 24 capas preentrenada sobre ImageNet. Reproducir esta red troncal desde cero y preentrenarla a continuación sobre ImageNet quedaba fuera del presupuesto de cómputo de este proyecto. ResNet18 (He et al., 2016) es el miembro más pequeño de la familia de redes residuales para el que torchvision distribuye un *checkpoint* preentrenado listo para usarse, y su *output stride* y profundidad de canales (reducción $\times 32$, 512 canales) encajan con una rejilla YOLO de $S = 7$ sobre una entrada de 448×448 . Aceptamos a cambio un tronco más pequeño, más rápido y ya preentrenado, a costa de una red de características menos expresiva que la del artículo; revisitamos esta compensación en la Discusión.

Adaptador. La capa `layer4` de ResNet18 produce mapas de características $(B, 512, 14, 14)$ para una entrada de 448×448 . La cabeza de detección necesita $(B, 1024, 7, 7)$ para coincidir con el tamaño esperado por la cabeza Darknet original. El adaptador es el módulo más barato que conecta ambos: una convolución de *stride* 2 reduce a la mitad la resolución espacial mientras dobla la profundidad de canales, y una convolución de *stride* 1 refina la representación resultante. Ambas convoluciones llevan *batch normalization* (Ioffe and Szegedy, 2015) y una activación LeakyReLU con pendiente negativa 0,1, igualando la elección del artículo para las capas de detección.

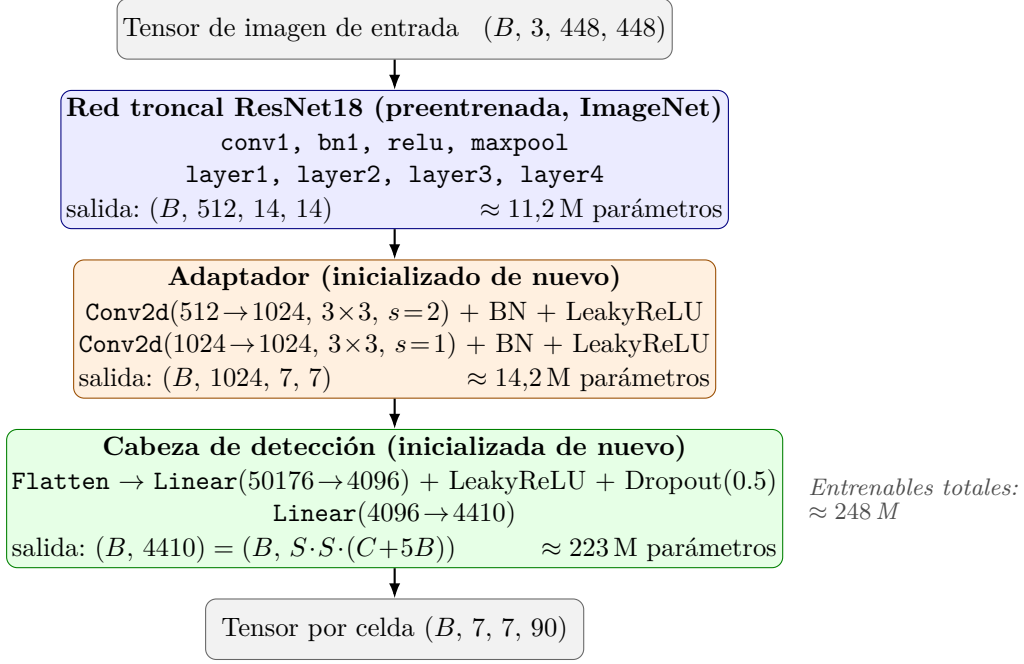


Figura 2: Arquitectura utilizada en V4–V6. Azul: tronco ResNet18 preentrenado cargado desde torchvision (`ResNet18_Weights.DEFAULT`). Naranja: adaptador convolucional que reduce espacialmente de 14×14 a 7×7 y ensancha la profundidad de canales de 512 a 1024. Verde: cabeza densa de dos capas; la primera capa densa ($50\,176 \rightarrow 4\,096$) acapara aproximadamente el 90 % de los parámetros entrenables del modelo. El tensor final se redimensiona a $(B, 7, 7, 90)$ para la pérdida y la decodificación; la disposición de canales por celda es la disposición de COCO con $C=80$ definida en la Sección 2.2.

Cabeza de detección. La cabeza es idéntica al artículo original. `Flatten` produce un vector de 50 176 elementos, la primera capa densa lo proyecta a 4 096 con LeakyReLU y *dropout* (Srivastava et al., 2014), y la capa densa final produce las $4\,410 = 7 \cdot 7 \cdot 90$ salidas. No se aplica activación a la capa final: la pérdida opera directamente sobre los *logits* crudos, sin sigmoide en las coordenadas de las cajas ni *softmax* en las probabilidades de clase.

Recuento de parámetros. La primera capa densa de la cabeza acapara aproximadamente 205 M de los 248 M parámetros entrenables ($50\,176 \times 4\,096$). El tronco ResNet18 aporta unos 11,2 M, el adaptador contribuye con ~ 14 M, y el resto se reparte entre la capa densa final y los diversos sesgos y términos de escala/desplazamiento de BN. La desproporción entre el tronco convolucional y la cabeza densa es una debilidad conocida del diseño de YOLOv1 y una de las motivaciones principales para la reescritura totalmente convolucional de YOLOv2 y YOLOv3 (Redmon and Farhadi, 2018).

3.3. Implementación de la pérdida

La pérdida es la formulación de suma de errores cuadrados con cinco términos de Redmon et al. (2016). La implementamos como una subclase única de `nn.Module` (`src/loss.py`) que opera sobre tensores por *batch*. En el Listado 1 se ofrece una versión condensada del cuerpo; discutimos cuatro detalles de implementación que no son visibles desde la formulación matemática.

```

1 predictions = predictions.reshape(-1, S, S, C + 5*B)
2
3 C = self.C
4 OBJ_IDX     = C
5 BOX1_SLICE  = slice(C + 1, C + 5)
6 CONF2_IDX   = C + 5
7 BOX2_SLICE  = slice(C + 6, C + 10)
8
9 iou_b1      = iou(predictions[..., BOX1_SLICE], target[..., BOX1_SLICE])
10 iou_b2      = iou(predictions[..., BOX2_SLICE], target[..., BOX1_SLICE])
11 best_box    = torch.max(torch.stack([iou_b1, iou_b2], dim=0), dim=0).indices
12
13 exists_box  = target[..., OBJ_IDX:OBJ_IDX+1]
14
15 # Pérdida de caja (xy directamente, wh bajo sign(.*sqrt(.)))
16 box_pred    = exists_box * (best_box * predictions[..., BOX2_SLICE]
17                             + (1 - best_box) * predictions[..., BOX1_SLICE])
18 box_tgt     = exists_box * target[..., BOX1_SLICE]
19
20 xy_pred     = box_pred[..., 0:2]
21 wh_pred_sqrt = torch.sign(box_pred[..., 2:4]) * torch.sqrt(
22                     torch.abs(box_pred[..., 2:4]) + 1e-6)
23 box_pred     = torch.cat([xy_pred, wh_pred_sqrt], dim=-1)
24 box_tgt     = torch.cat([box_tgt[..., 0:2],
25                         torch.sqrt(box_tgt[..., 2:4] + 1e-6)], dim=-1)
26 box_loss    = mse(box_pred, box_tgt)
27
28 # Pérdidas de objeto, no-objeto y clase (ver src/loss.py para el código completo)

```

Listing 1: Cuerpo de `YoloLoss.forward`, abreviado para no extender la exposición. Las variables auxiliares de índices y la máscara de responsabilidad basada en IoU se omiten; el código completo está en `src/loss.py`.

Reducción por suma. Usamos `nn.MSELoss(reduction="sum")` y no "mean". El artículo especifica una suma de errores cuadrados, y la ponderación relativa entre los cinco términos está calibrada para reducción por suma: $\lambda_{\text{coord}} = 5$ es significativo solo si todos los sumandos de coordenadas se acumulan en lugar de promediarse. La consecuencia es que la magnitud absoluta de la pérdida escala con el tamaño de *batch* y con la densidad de la rejilla; con nuestros valores $S = 7$, $B = 2$, $C = 80$ y *batch* 32, la pérdida por *batch* se sitúa en el orden de las centenas (como se observa en V5 y V6 en la Sección 4.1).

Raíz cuadrada segura sobre anchura y altura. El artículo aplica $\sqrt{w}$ y $\sqrt{h}$ para equilibrar la contribución de gradiente de cajas pequeñas y grandes. La salida de la red no está restringida, sin embargo, por lo que cualquier predicción negativa en fase temprana del entrenamiento haría fallar la raíz cuadrada. Por tanto calculamos $\text{sign}(x) \cdot \sqrt{|x| + \varepsilon}$ con $\varepsilon = 10^{-6}$. El signo se conserva para que el gradiente a través de la máscara de responsabilidad apunte de vuelta hacia el semieje positivo en el paso siguiente, en lugar de quedar atrapado por la simetría del valor absoluto.

Este truco es estándar en reimplementaciones publicadas de YOLO, pero la elección de dónde colocar ε importa: $\sqrt{|x| + \varepsilon}$ es más seguro que $\sqrt{|x + \varepsilon|}$ en las cercanías de $x = -\varepsilon$.

Evitar escrituras *in-place* sobre tensores con autograd. Una versión anterior del código transformaba `box_predictions[..., 2:4]` *in-place*:

```
1 box_predictions[..., 2:4] = torch.sign(...) * torch.sqrt(...)
```

Esto rompe silenciosamente el grafo de autograd a través del *slice* modificado: no se levanta excepción, el valor de la pérdida es correcto, pero la pasada hacia atrás por el camino w, h devuelve gradientes cero. La implementación actual reconstruye el tensor de la caja de cuatro componentes mediante `torch.cat` a lo largo del canal, lo que preserva el grafo. Esto se corrigió en el mismo ciclo de *commits* que la auditoría de la Sección 4.6; lo mencionamos aquí porque forma parte de los invariantes de implementación que las modificaciones futuras deben respetar.

Índices derivados de C . Como se discute extensamente en la Sección 4.6.1, los índices al tensor de predicción por celda se calculan a partir de `self.C` en lugar de quedar literales. La misma clase puede instanciarse para cualquier C sin cambios de código, lo cual corrige el error original sobre COCO y mantiene la clase testeable sobre la disposición VOC de $C = 20$ sin modificaciones.

3.4. Infraestructura de entrenamiento

El bucle de entrenamiento (`src/train.py`) está escrito para operación multi-GPU bajo `DistributedDataParallel` (DDP) de PyTorch y se lanza con `torchrun`. En la práctica, las seis ejecuciones de este trabajo usaron una sola GPU, pero el andamiaje multi-proceso (inicialización del grupo de procesos, *sampler* distribuido, log y *checkpoint* únicamente desde el rango cero) se conservó para poder escalar el bucle más adelante sin reestructurarlo.

Hardware. V4–V6 se ejecutaron en un *pod* de Vast.ai con una única NVIDIA H200 NVL (140 GB HBM3) y almacenamiento NVMe local al nodo. El *throughput* con $S = 7$, $B = 2$, $C = 80$, *batch* 32 y entrada 448×448 fue de aproximadamente 114 segundos por época sobre el conjunto de entrenamiento, es decir ~ 95 minutos para 50 épocas. V1–V3 corrieron en un proveedor distinto (RunPod) sobre un *pod* equipado con ocho GPUs NVIDIA H100, de las cuales solo una se utilizó efectivamente porque el sustrato de almacenamiento se convirtió en cuello de botella antes de poder explotar el paralelismo de cómputo; la configuración se discute en “Sustrato de almacenamiento, de paso” más abajo. Los tiempos absolutos de V1–V3 no son directamente comparables con los de V4–V6 porque la red troncal, el número de iteraciones, la generación de GPU y la ruta de E/S difieren todos.

Pila de software. PyTorch 2.4.1 con CUDA 12.4 sobre Python 3.12. La precisión mixta se gestiona con `torch.amp.autocast(device_type='cuda', dtype=torch.float16)` para la pasada hacia adelante y la pérdida, mientras el optimizador mantiene una copia maestra de pesos en *float32* mediante `torch.amp.GradScaler`. Los gradientes se desescalan antes del recorte para que el umbral se interprete en magnitud real y no en unidades escaladas.

Optimizador y planificador. Descenso por gradiente estocástico con momento 0,9 y *weight decay* 5×10^{-4} , aplicado uniformemente a todos los parámetros (no usamos grupos de parámetros con tasa de aprendizaje reducida sobre la red troncal). El planificador es un calentamiento lineal de tres épocas desde 0 hasta la tasa pico, seguido de *cosine annealing* hasta 10^{-5} (Loshchilov and Hutter, 2017). La tasa pico es 10^{-4} en V4–V6 y varía a través de V1–V3 como se describe en la Sección 4.1.

Checkpoints. El *trainer* escribe tres tipos de *checkpoint* por ejecución: un `last.pth` rotativo actualizado cada época, una instantánea `epoch_NNN.pth` cada dos épocas y un `best.pth` actualizado solo cuando la pérdida de validación mejora estrictamente. Cada *checkpoint* “completo” contiene el *state dict* del modelo, el estado del optimizador, el estado del *AMP scaler*, el número de época y las pérdidas de entrenamiento/validación, de modo que una ejecución puede retomarse de forma exacta (así es como V6 retoma desde V5). Para la evaluación aceptamos tanto *checkpoints* “completos” como “limpios” que son *state dicts* puros; la distinción se detecta por la presencia de la clave `model_state_dict` y se maneja en `scripts/evaluate.py` y `scripts/visualize_prediction.py`.

Sustrato de almacenamiento, de paso. Un intento previo de alojar V1–V3 sobre un *pod* de RunPod equipado con 8× GPUs NVIDIA H100 y almacenamiento distribuido respaldado por MooseFS se abandonó después de que el paso de descompresión de COCO colocara repetidamente procesos de trabajo en suspensión ininterrumpible (estado D), exigiendo reinicios del *pod* e impidiendo que la configuración de ocho GPUs llegara a ejercitarse en una ejecución real de entrenamiento. La lección relevante es que un dataset de 26 GB de ficheros pequeños se beneficia sustancialmente de un NVMe local frente a un sistema de ficheros en red: cambiar a un NVMe local del nodo (que el *pod* de Vast.ai utilizado para V4–V6 proporciona de fábrica) fue la mejora de ingeniería más impactante del proyecto, más impactante que cualquier cambio arquitectónico entre V1 y V4.

3.5. Pipeline de evaluación

La evaluación se realiza fuera de línea con `scripts/evaluate.py` contra el conjunto de prueba. El script carga un *checkpoint*, itera sobre el `DataLoader` de prueba con `shuffle=False` (para que los índices contables sean deterministas), y para cada *batch* ejecuta los siguientes pasos.

1. Pasada hacia adelante bajo `torch.no_grad`, produciendo el tensor de salida plano ($B, 4410$).
2. Decodificación por celda mediante `utils.cellboxes_to_boxes`, que redimensiona a $(B, 7, 7, 90)$, selecciona la caja candidata de mayor confianza de las dos por celda, aplica la transformación celda-imagen discutida en la Sección 4.6.2 (el factor $1/S$ sobre x, y únicamente, tras la corrección), y devuelve una lista de 49 cajas candidatas por imagen en formato `[clase, prob, x, y, w, h]`.
3. *Non-maximum suppression* por clase, parametrizado por un umbral de probabilidad (la menor confianza predicha a considerar; fijado a 0,1 y 0,3 en los resultados que reportamos)

y un umbral de IoU (el solapamiento por encima del cual se descarta una predicción de la misma clase con menor confianza; fijado a 0,5).

4. Agregación de todas las predicciones supervivientes a lo largo del conjunto de prueba completo en una lista global de tuplas `[idx_imagen, clase, prob, x, y, w, h]`.
5. Extracción de la lista correspondiente de cajas de referencia a partir de los tensores de *target*, también indizada por `idx_imagen`.
6. Cálculo del mean Average Precision mediante `metrics.mean_average_precision`, que ordena las predicciones de cada clase por confianza, las recorre en orden descendente marcando cada una como verdadero positivo (si solapa una caja de referencia no reclamada de la misma clase con IoU por encima del umbral) o falso positivo, e integra la curva de precisión-*recall* resultante por clase mediante la regla del trapecio. El mAP final es la media de los AP por clase sobre las 80 clases.

Un detalle de implementación. El código de mAP descarta las clases con cero instancias de referencia en el conjunto evaluado, en lugar de contarlas como $AP = 0$. Sobre COCO esto nunca se activa, porque cada clase tiene al menos una instancia en el conjunto de prueba, pero cambiaría el denominador si el mismo código se reutilizara sobre un *benchmark* más pequeño.

Reproducibilidad. La partición del dataset queda fijada por la semilla 42 en `scripts/prepare_splits.py`, el `DataLoader` de prueba usa `shuffle=False`, NMS es determinista, y el código de pérdida y decodificación no contiene elementos estocásticos. Volver a ejecutar el script de evaluación sobre el mismo *checkpoint* y el mismo hardware reproduce el mAP reportado de forma exacta.

4. Experimentación

Esta sección recorre seis ejecuciones sucesivas de entrenamiento (V1–V6) de la implementación de YOLOv1 sobre COCO 2017. Las cuatro primeras produjeron $mAP@0,5 = 0$ sobre el conjunto de prueba reservado pese a que las pérdidas de entrenamiento parecían converger. La salida de este estado de cosas es la narrativa central de este trabajo: una secuencia de cambios de hiperparámetros que no diagnosticaron el problema, seguida de una auditoría de código que identificó dos errores de implementación invisibles para la batería de tests. V5 y V6 se entrenaron tras las correcciones y produjeron las primeras puntuaciones de detección no nulas.

Adoptamos la convención de identificar cada versión por su etiqueta corta (V1...V6) y referenciamos cruzadamente la fila correspondiente de la Tabla 1 a lo largo del texto.

4.1. Configuración común de entrenamiento

Las seis versiones comparten el optimizador, la formulación de la pérdida (cuya implementación, sin embargo, difería en V1–V4 respecto a V5–V6 debido a las correcciones discutidas en la Sección 4.6), el pipeline de datos y el protocolo de evaluación. Aquí resumimos las constantes y describimos solo las variaciones por versión en sus respectivas subsecciones.

- **Dataset:** COCO 2017 (Lin et al., 2014), las 80 clases, consolidado en un único fichero de anotaciones y particionado de forma determinista (semilla 42) en 90 % / 5 % / 5 % para entrenamiento, validación y prueba. El conjunto de prueba contiene 6 165 imágenes y 42 795 cajas delimitadoras de referencia tras la eliminación de las anotaciones de tipo *crowd*. De estas, la clase de dataset solo retiene 31 167 en tiempo de entrenamiento porque la regla de un objeto por celda (Sección 3.1) descarta cualquier anotación cuya celda en la rejilla ya esté reclamada por un objeto encontrado previamente; las cajas descartadas siguen contando como detecciones perdidas en la evaluación, dado que el evaluador lee las anotaciones COCO crudas.
- **Optimizador:** descenso por gradiente estocástico con momento sin Nesterov igual a 0,9 y *weight decay* 5×10^{-4} , aplicado uniformemente a todos los parámetros (sin grupos de parámetros).
- **Planificador de tasa de aprendizaje:** calentamiento lineal de tres épocas desde 0 hasta la tasa pico, seguido de *cosine annealing* (Loshchilov and Hutter, 2017) hasta un mínimo de 1×10^{-5} . La tasa pico es el parámetro que varía por versión y se reporta individualmente.
- **Precisión mixta:** la pasada hacia adelante corre bajo `torch.amp.autocast` en *float16*, mientras los pesos maestros y el acumulador de pérdida permanecen en *float32* (Micikevicius et al., 2018). El escalado de gradiente se realiza con `GradScaler` y los gradientes escalados se desescalan antes del recorte.
- **Configuración distribuida:** el bucle de entrenamiento está escrito para `DistributedDataParallel` con `torchrun`, pero en la práctica todas las ejecuciones usaron una sola GPU con un tamaño de *batch* por GPU de 32. V1–V3 corrieron en un *pod* de RunPod equipado con 8× GPUs NVIDIA H100 (de las cuales solo una se ejercitó efectivamente debido al cuello de botella de almacenamiento descrito en la Sección 3.4); V4–V6 corrieron en un *pod* de Vast.ai con una NVIDIA H200 NVL (140 GB HBM3) y NVMe local al nodo.
- **Pérdida:** la suma de errores cuadrados de cinco términos de Redmon et al. (2016), con $\lambda_{\text{coord}} = 5$ y $\lambda_{\text{noobj}} = 0,5$. La implementación es objeto de la Sección 4.6.
- **Evaluación:** el modelo entrenado se ejecuta sobre el conjunto de prueba completo con umbrales de probabilidad de 0,1 y 0,3, seguidos de *non-maximum suppression* por clase a IoU 0,5. La métrica reportada es el mean Average Precision a IoU 0,5, calculada por clase con el sistema de registro de cajas reclamadas no descritas en la Sección 2.5 y promediada sobre las 80 clases que tienen al menos una instancia de referencia en el conjunto de prueba (en nuestro caso, todas).

Las variables que cambian entre versiones son la red troncal, la tasa de aprendizaje pico, el umbral de recorte de gradiente, el número de épocas de entrenamiento y, lo más importante para V5 y V6, el código de pérdida y decodificación. Se resumen en la Tabla 1.

Tabla 1: Resumen de las seis ejecuciones de entrenamiento. “Errores” indica si los errores de pérdida y decodificación de coordenadas analizados en la Sección 4.6 estaban presentes en tiempo de entrenamiento. La pérdida de validación es el valor final por *batch* con reducción por suma del registro de entrenamiento; el salto de magnitud entre V4 y V5 es por sí mismo evidencia de la corrección y se discute en la Sección 4.7.

Run	Red troncal	Errores	Épocas	Train loss	Val loss	mAP@0.5 (0.1)	mAP@0.5 (0.3)
V1	Darknet24	presentes	33 (diverge)	–	N/A	0.0000	0.0000
V2	Darknet24	presentes	100	1.45	1.50	0.0000	0.0000
V3	Darknet24	presentes	135	1.20	1.35	0.0000	0.0000
V4	ResNet18	presentes	50	0.96	0.98	0.0000	0.0000
V5	ResNet18	corregidos	50	336.34	337.37	0.0409	0.0373
V6	ResNet18	corregidos	60	330.50	337.40	0.0415	0.0381

4.2. V1: línea base fiel al artículo con Darknet

El primer intento sigue a Redmon et al. (2016) de la forma más literal que permite una pila moderna de PyTorch (Paszke et al., 2019). La red troncal es una reimplementación del Darknet de 24 capas del artículo, inicializada desde cero con la inicialización por defecto de PyTorch. La cabeza de detección son dos capas totalmente conectadas ($50\,176 \rightarrow 4\,096 \rightarrow 4\,410$) con *dropout* 0,5 tras la primera (Srivastava et al., 2014). La tasa de aprendizaje pico es 1×10^{-3} , igualando la elección del artículo para entrenamiento desde cero sin pretrain de ImageNet. No se aplica recorte de gradiente. El calendario previsto es de 100 épocas.

El entrenamiento diverge en la época 33: el tensor de pérdida se convierte en NaN dentro de un solo *batch* y permanece como NaN en cada paso posterior. La causa inmediata es la combinación de una red convolucional de 24 capas inicializada al azar, una tasa de aprendizaje pico alta y la ausencia de cualquier mecanismo de recorte: un único *batch* desafortunado produce una norma de gradiente lo bastante grande para empujar los pesos maestros a una región donde el siguiente *forward* desborda el rango dinámico de *float16*. La ejecución se abandona y no se selecciona ningún *checkpoint* para evaluación. Incluimos no obstante V1 en la Tabla 1 por completitud narrativa.

4.3. V2: regularización mediante recorte de gradiente

V2 es una respuesta directa a V1: la tasa pico se reduce a la mitad (5×10^{-4}) e introducimos recorte de norma de gradiente con umbral 5,0. La arquitectura permanece sin cambios. El modelo entrena durante las 100 épocas previstas sin divergencia; la pérdida de validación toca fondo en torno a 1,50 y se estanca. Sobre el conjunto de prueba, no obstante, el mAP@0.5 sigue siendo exactamente cero. El comportamiento cualitativo es el de un modelo que ha aprendido a llevar las confianzas de objeto a casi cero en todas partes: las detecciones tras NMS son escasas y sus cajas no guardan relación con los objetos de referencia.

Interpretamos esto en su momento como signo de que la tasa de aprendizaje reducida estaba impidiendo a la red escapar de un mínimo trivial del término de confianza sin objeto. En retrospectiva, esa interpretación era incompleta: el “mínimo trivial” era real, pero su existencia la causaba el error de pérdida discutido en la Sección 4.6.1, no los hiperparámetros.

4.4. V3: relajar el recorte, restaurar la tasa

V3 devuelve la tasa de aprendizaje pico a 1×10^{-3} y sube el umbral de recorte de gradiente a 10,0, con la hipótesis de trabajo de que V2 estaba subentrenado. El número de épocas se extiende a 135 para igualar la duración total del calendario del artículo. La pérdida de entrenamiento sigue descendiendo y alcanza su valor más bajo entre las ejecuciones con Darknet (1,20), pero la pérdida de validación empieza a divergir de la de entrenamiento alrededor de la época ~ 40 , lo que interpretamos como sobreajuste. El mAP@0.5 sobre el conjunto de prueba es de nuevo exactamente cero.

En ese punto, la hipótesis de trabajo pasó a ser que la red troncal Darknet, entrenada desde cero sobre COCO sin el preentrenamiento sobre ImageNet que realizó Redmon et al. (2016), sencillamente carece del sesgo inductivo necesario para este dataset. Pasamos entonces a V4.

4.5. V4: ResNet18 preentrenada sobre ImageNet

V4 reemplaza el Darknet de 24 capas por una ResNet18 (He et al., 2016) cargada con los pesos de ImageNet desde `torchvision (ResNet18_Weights.DEFAULT)`. La cabeza de clasificación de la ResNet se descarta y el tronco convolucional hasta `layer4` incluido produce mapas de características de forma $(B, 512, 14, 14)$ para la entrada estándar de 448×448 . Hacemos de puente hacia el $(B, 1024, 7, 7)$ que espera la cabeza de detección con un adaptador de dos capas (una convolución de *stride* 2 de 512 a 1024 canales, seguida de una convolución de *stride* 1 que refina la representación), cada una con *batch normalization* (Ioffe and Szegedy, 2015) y una activación LeakyReLU. La cabeza de detección es idéntica a V1–V3. Todos los parámetros del modelo llevan `requires_grad=True` desde la primera época; no congelamos ninguna capa.

Como la red troncal espera ahora entradas normalizadas según ImageNet, la transformación del dataset se amplía con la normalización estándar canal a canal por la media y la desviación. La tasa pico se reduce a 1×10^{-4} para hacer *fine-tuning* de las características convolucionales preentrenadas sin destruirlas, dejando al mismo tiempo capacidad de aprendizaje suficiente al adaptador y a la cabeza inicializados al azar (que entre los dos acaparan la mayoría de los 248 M parámetros del modelo). El recorte de gradiente se mantiene en 10,0 y el calendario se acorta a 50 épocas.

V4 entrena sin incidencias numéricas. La pérdida de validación alcanza 0,98 en la mejor época, el valor más bajo hasta ahora y, en apariencia, sugestivo de un modelo bien ajustado a los datos. Sin embargo el mAP@0.5 sobre el conjunto de prueba vuelve a ser exactamente cero. Tras NMS observamos un modo de fallo cualitativo distinto al de V2/V3: el modelo emite gran cantidad de cajas de baja confianza por imagen (del orden de 20 con $p > 0,005$), pero están espacialmente descorrelacionadas con la referencia, y sus clases predichas no coinciden con las clases correctas.

Este fue el resultado que forzó la pregunta: *¿está bajando la pérdida porque el modelo está aprendiendo, o porque está encontrando un mínimo trivial de una pérdida que no es la que pensamos que es?* La investigación que siguió es el objeto de la siguiente sección.

4.6. Investigación de errores

El disparador de una auditoría fue la observación de que, a lo largo de cuatro ejecuciones con configuraciones de hiperparámetros muy distintas y dos redes troncales diferentes, el mAP de prueba permanecía idénticamente en cero mientras las pérdidas de validación cubrían el rango 0,96–1,50. Una pérdida que decrece de forma consistente a lo largo de barridos de hiperparámetros pero que produce un detector que no detecta nada es sospechosa: o bien el modelo carece realmente de capacidad de localización en ese régimen (lo cual cuesta reconciliar con una ResNet18 plenamente *fine-tuned* con $\text{val_loss} < 1$), o bien la pérdida no está imponiendo la restricción que esperamos.

La auditoría identificó dos errores independientes, cada uno suficiente por sí solo para producir $\text{mAP@0,5} = 0$. El primero está en la función de pérdida; el segundo en la rutina de post-procesamiento que convierte las salidas del modelo en cajas relativas a la imagen. Los discutimos por turno.

4.6.1. Error 1: índices de VOC aplicados a tensores de COCO

La implementación original de la pérdida YOLO en `src/loss.py` indizaba el tensor de predicción por celda mediante enteros literales que correspondían a la disposición de Pascal VOC (20 clases, $C = 20$). La disposición esperada por celda para $C = 20$ es

$$\underbrace{[0, \dots, 19]}_{\text{prob. de clase}} \quad \underbrace{[20]}_{\text{conf}_1} \quad \underbrace{[21, \dots, 24]}_{x_1, y_1, w_1, h_1} \quad \underbrace{[25]}_{\text{conf}_2} \quad \underbrace{[26, \dots, 29]}_{x_2, y_2, w_2, h_2},$$

que la pérdida lee literalmente como `[..., 20:21]` para la máscara “existe objeto”, `[..., 21:25]` para la primera caja predicha y así sucesivamente. Para COCO con $C = 80$, el tensor real por celda que produce el dataset es

$$\underbrace{[0, \dots, 79]}_{\text{prob. de clase}} \quad \underbrace{[80]}_{\text{conf}_1} \quad \underbrace{[81, \dots, 84]}_{x_1, y_1, w_1, h_1} \quad \underbrace{[85]}_{\text{conf}_2} \quad \underbrace{[86, \dots, 89]}_{x_2, y_2, w_2, h_2},$$

por lo que los índices que usa la pérdida apuntan dentro de la región de *probabilidades de clase* del tensor, sin llegar nunca a los huecos de coordenadas.

Las consecuencias aguas abajo son catastróficas pero localmente silenciosas:

1. `target[..., 20:21]`, que la pérdida interpreta como indicador de presencia de objeto (`exists_box`), es en COCO el hueco de probabilidad de clase para el índice 20 (que, tras la reasignación densa de COCO, corresponde a la clase *elephant*). La máscara vale 1 únicamente en celdas cuya clase de referencia es *elephant*, y 0 en el resto.
2. Los huecos de coordenadas que la pérdida lee (`predictions[..., 21:25]`, `target[..., 21:25]`) son en realidad huecos de probabilidad de clase para los índices 21–24 (*bear*, *zebra*, *giraffe*, *backpack*). El cálculo de IoU entre dos rebanadas de probabilidades de clase devuelve una cantidad sin sentido prácticamente sin correlación con nada que la red pueda aprender.

3. La pérdida de clase se calcula sobre la rebanada `predictions[..., :20]`, por lo que solo las primeras veinte clases de COCO (de ochenta) participan en el término de clase. Las clases 20–79 no reciben señal alguna de gradiente.
4. La pérdida sin objeto penaliza `predictions[..., 20]` y `predictions[..., 25]` en celdas donde la posición 20 del *target* es cero. La posición 20 del *target* es cero en la inmensa mayoría de las celdas (las que no contienen un elefante), por lo que el modelo se ve fuertemente empujado a llevar esas dos posiciones de salida a cero en todas partes. Las posiciones 20 y 25 del tensor de predicción, sin embargo, no son los huecos de confianza; son los huecos de probabilidad de clase para *elephant* y (en la práctica, tras `argmax`) el hueco interpretado como “conf 2” es la probabilidad de clase para *train*. El comportamiento de mínimo trivial observado en V2–V4 es un aprendizaje fiel de esta señal confundida de no-objeto, no una propiedad de la pérdida MSE como tal.

El error es invisible desde el punto de vista de las pruebas unitarias. El test existente en `tests/test_loss.py` instancia la pérdida con $C = 20$:

```

1 loss_fn = YoloLoss(S=7, B=2, C=20)
2 target = torch.zeros(1, 7, 7, 30)
3 target[0, 3, 4, 0]      = 1                # clase
4 target[0, 3, 4, 20]     = 1                # confianza
5 target[0, 3, 4, 21:25] = torch.tensor([0.5, 0.5, 0.3, 0.3])

```

Listing 2: Test unitario previo a la corrección que pasaba por la razón equivocada.

Con $C = 20$, los índices 20, 21:25, 25, 26:30 que la pérdida lee coinciden con los huecos donde el dataset escribe, por lo que el test pasa. El test tendría que estar parametrizado sobre C para detectar el desajuste; el nuestro no lo estaba.

Verificación. Construimos un *target* de forma COCO con un único objeto de referencia de clase 7, en la celda (3, 4), con caja $(x_c, y_c, w, h) = (0, 5, 0, 5, 0, 3, 0, 3)$ y calculamos la pérdida para dos predicciones:

1. Una predicción “perfecta” que iguala el *target* en todos los huecos, incluidos los huecos de coordenadas COCO correctos [81:85].
2. La misma predicción pero con los cuatro huecos de coordenadas [81:85] sobrescritos con valores arbitrariamente grandes (99, 0, ..., 99, 0).

Bajo una pérdida correctamente indexada, la predicción (2) debe producir un valor mucho mayor que la predicción (1). Bajo la pérdida con error, los dos valores eran idénticos hasta la precisión numérica: 0,000490 en ambos casos. Esto confirma que la pérdida era *ciega al gradiente* de los huecos que el modelo se supone que debe predecir.

Agregado sobre una muestra de 200 imágenes reales de COCO de entrenamiento, la máscara `exists_box` con error valía 1 únicamente en 3 celdas de las 1 062 que realmente contenían un objeto de referencia (alrededor del 0,3 %), y las tres eran elefantes. El 99,7 % restante de los objetos reales contribuía exactamente cero a los términos de caja y objeto de la pérdida.

Corrección. Sustituimos cada literal codificado en la pérdida por un índice derivado de `self.C`:

```

1 C          = self.C
2 OBJ_IDX    = C
3 BOX1_SLICE = slice(C + 1, C + 5)
4 CONF2_IDX  = C + 5
5 BOX2_SLICE = slice(C + 6, C + 10)

```

Listing 3: Variables de índice introducidas al inicio de `YoloLoss.forward`.

y propagamos estos nombres por cada sitio donde antes había un literal. El término de clase se corrigió también de `predictions[..., :20]` a `predictions[..., :C]` para que las 80 clases participen. El test unitario preexistente con $C = 20$ sigue pasando porque las nuevas variables evalúan a las mismas posiciones numéricas cuando $C = 20$. El *diff* completo lado a lado se proporciona en el Apéndice B.

Un segundo seguimiento, más pequeño, se aplicó a la pérdida poco después: la transformación de raíz cuadrada de w y h (el truco del artículo para igualar la contribución de cajas pequeñas y grandes) estaba implementada como una asignación *in-place*, `box_predictions[..., 2:4] = ....` Las escrituras *in-place* sobre un tensor que participa en el grafo de autograd rompen silenciosamente la pasada hacia atrás a través de ese tensor, sin que se levante excepción en tiempo de ejecución. La corrección es reensamblar el tensor modificado con `torch.cat` a lo largo del canal. Lo mencionamos aquí por completitud; no afectó a las ejecuciones V1–V4 porque, dado el Error 1, ningún gradiente fluía por ese camino de todas formas.

4.6.2. Error 2: factor $1/S$ espurio en la decodificación de w, h

La rutina de post-procesamiento `convert_cellboxes` en `src/utils.py` convierte el tensor de predicción crudo por celda en una lista de cajas en coordenadas relativas a la imagen. La conversión de las coordenadas del centro (x, y) es directa: las predicciones se almacenan como desplazamientos dentro de la celda, por lo que la coordenada x global es el índice de columna de la celda más el desplazamiento intra-celda, dividido por S :

$$x_{\text{global}} = \frac{j + x_{\text{cell}}}{S}, \quad y_{\text{global}} = \frac{i + y_{\text{cell}}}{S}.$$

La anchura y la altura, sin embargo, las almacena el dataset como cocientes de las dimensiones de la *imagen completa* ($w_{\text{norm}} = w_{\text{px}}/W_{\text{img}}$ y análogamente para h), y la pérdida compara la salida w, h del modelo *directamente* contra estos valores. El modelo, por tanto, aprende a predecir $w_{\text{norm}}, h_{\text{norm}}$.

En nuestra implementación, no obstante, el código de decodificación ejecutaba una división adicional por S :

```

1 w_h = (1.0 / S) * best_boxes[..., 2:4]  # BUG: divide w, h por 7

```

Este es el error simétrico que aparece al portar desde una codificación distinta. La implementación de la que tomamos inspiración parcial (Persson, 2020) almacena anchura y altura como $w_{\text{norm}} \cdot S$ y $h_{\text{norm}} \cdot S$ (“número de celdas que la caja abarca”), por lo que dividir por S en la decodificación es correcto *en ese código*. En el nuestro no.

Consecuencia. Cada caja predicha se decodifica con anchura y altura siete veces menor que el valor que el modelo ha aprendido a emitir. Dado que la red estaba entrenada para predecir $w_{\text{norm}}, h_{\text{norm}} \in [0, 1]$, este reescalado produce cajas de a lo sumo un séptimo del tamaño lineal correcto y aproximadamente un cuadragésimo noveno del área correcta.

La cota aritmética superior de la IoU entre tal predicción encogida y su caja de referencia perfectamente alineada es

$$\text{IoU} \leq \frac{(\frac{1}{S})^2}{1 + (\frac{1}{S})^2 - (\frac{1}{S})^2} = \frac{1}{S^2} \approx 0,020,$$

muy por debajo del umbral de 0,5 usado en mAP@0.5. El error es por tanto suficiente por sí solo para llevar la métrica a exactamente cero, incluso en el escenario hipotético en el que el error de pérdida no hubiera existido y la red hubiera aprendido a localizar objetos correctamente.

Verificación. Codificamos una caja conocida con $w = h = 0,3$ centrada en la celda $(3, 3)$, pasamos el tensor por `convert_cellboxes` e inspeccionamos la anchura y altura decodificadas. El código con error devolvía $w_{\text{dec}} = h_{\text{dec}} = 0,0429$, que es exactamente $0,3/7$, mientras x e y hacían *round-trip* correctamente. Tras eliminar el factor $1/S$ sobre w_h , el *round-trip* es exacto hasta la precisión de punto flotante.

Corrección. La corrección es de una línea: eliminar el factor $1/S$ sobre w_h manteniéndolo sobre x e y . Se renombra la variable de `w_y` a `w_h` para mayor claridad.

4.6.3. Por qué la batería de tests no detectaba ninguno de los dos errores

Ambos errores son detectables con un único chequeo de *round-trip*, pero la batería existente cubría únicamente formas y rangos de valor. Los tests preexistentes para `convert_cellboxes` eran:

- `test_convert_cellboxes_output_shape`: asegura que la salida tiene la forma $(B, S^2, 6)$.
- `test_convert_cellboxes_class_indices_in_range`: asegura que la columna de índice de clase predicho cae en $[0, C - 1]$.

Ninguno inspecciona el valor de w, h . Un test de la forma “codifica una caja conocida, decodifícala, asegura que las w, h recuperadas coinciden” habría detectado el Error 2 de manera trivial. De forma análoga, un test de pérdida parametrizado sobre C con $C \in \{20, 80\}$ habría detectado el Error 1 en la primera ejecución: el test de “predicción perfecta” con $C = 80$ habría devuelto una pérdida no nula para cualquier predicción cuyos valores en los huecos VOC espurios no coincidieran casualmente con las probabilidades de clase en esos huecos. Añadimos ambos tests de *round-trip* junto con las correcciones, como documenta la Sección 4.7.

Lección generalizable. El fallo más profundo aquí no son los índices concretos sino la suposición de que un test que compila, se ejecuta y pasa constituye evidencia de que la función que prueba es correcta. Los dos tests anteriores hacían exactamente lo que afirmaban y aportaban cero evidencia sobre la corrección matemática de la función. En un pipeline de detección, donde

los tensores intermedios tienen muchos huecos y la mayoría no se utiliza en una única imagen de test, la densidad de cobertura significativa por test es baja. Los chequeos de *round-trip* que añadimos fijan un único invariante cuantitativo por función y se rompen cuando el contrato matemático de la función cambia, que es el nivel de especificidad que falta en las aserciones de solo forma.

4.7. V5: aplicar las correcciones

V5 mantiene todos los hiperparámetros de V4 — la misma red troncal ResNet18, el mismo adaptador, la misma cabeza FC, la misma tasa de aprendizaje pico de 1×10^{-4} , el mismo umbral de recorte de gradiente de 10,0, el mismo calendario de 50 épocas, el mismo optimizador, tamaño de *batch* y configuración de calentamiento — y cambia solo el código de pérdida y de decodificación tal y como describe la Sección 4.6. El sentido de mantener todo lo demás igual es atribuir cualquier cambio en el resultado únicamente a las correcciones.

La primera observación, disponible desde la primera época de entrenamiento, es que la magnitud de la pérdida ahora vive en un régimen completamente distinto. Mientras V4 producía pérdidas de entrenamiento y validación en torno a 0,96–0,98, V5 produce valores en el rango de varias centenas. Las pérdidas finales de entrenamiento y validación son 336,34 y 337,37 respectivamente. Esperábamos este comportamiento: la pérdida se reduce con `nn.MSELoss(reduction="sum")` sobre los 90 huecos de cada celda de cada imagen del *batch*. En V1–V4, debido al Error 1, casi todos los sumandos por celda colapsaban a cero porque se multiplicaban por la máscara *exists_box*, que era cero casi en todas partes. Las contribuciones no trivialmente nulas procedían casi por completo de unas pocas celdas de elefantes. Una vez corregidos los índices, cada celda con un objeto contribuye a los términos de caja y de confianza, y cada celda contribuye al término sin objeto y al de clase. La magnitud absoluta de la pérdida aumenta por tanto en dos órdenes de magnitud. Esto no es una regresión: es la pérdida viendo finalmente los datos.

En tiempo de evaluación, V5 produce $\text{mAP}@0,5 = 0,0409$ con umbral de probabilidad 0,1 sobre el conjunto de prueba, y 0,0373 con umbral 0,3 (véase la Tabla 1). Esta es la primera puntuación de detección no nula de todo el proyecto. Es un número absoluto bajo — discutimos en detalle en la Sección 5 por qué lo es, y en la Discusión qué sería necesario para subirlo — pero es un cambio categórico respecto a las cuatro ejecuciones anteriores.

El comportamiento cualitativo también cambia. Las cajas de detección se agrupan ahora sobre objetos reales de la escena, las clases predichas se correlacionan con las clases de referencia sobre las clases fáciles (animales de silueta distintiva, vehículos), y el número de detecciones tras NMS desciende a un sensato 2–6 cajas por imagen con umbral 0,3. Ofrecemos ejemplos extraídos del conjunto de prueba, ordenados por calidad cualitativa, en la Sección 5.

4.8. V6: continuación de V5

V6 es una continuación de V5 más que un experimento separado. Reanudamos el entrenamiento desde el *checkpoint* “best” de V5 y entrenamos durante 10 épocas adicionales, manteniendo todos los demás ajustes idénticos. La motivación fue verificar que el resultado de V5 no era casualidad de la fase del coseno en una época favorable y dar al modelo un poco más de tiempo cerca del valle de la tasa de aprendizaje.

La pérdida de validación final es esencialmente idéntica a la de V5 (337,40 frente a 337,37), lo que indica que el planificador ha alcanzado saturación. El mAP de prueba mejora marginalmente a 0,0415 con umbral de probabilidad 0,1 y 0,0381 con 0,3. Tratamos a V6 como la ejecución titular del proyecto y usamos su *checkpoint* (`v6_run/best_v6.pth`) para todos los análisis posteriores en la Sección 5, incluyendo la tabla completa de AP por clase en el Apéndice A y las figuras cualitativas.

4.9. Curvas de pérdida y una nota sobre el salto de magnitud

La Figura 3 representa las curvas de pérdida de entrenamiento y validación a lo largo de V4, V5 y V6 sobre un eje temporal compartido. El salto visual en magnitud absoluta entre V4 y V5 es la firma empírica de la corrección de índices. El panel de la derecha hace un *zoom* sobre V5+V6 para hacer visible el descenso lento de ambas pérdidas a la escala apropiada.

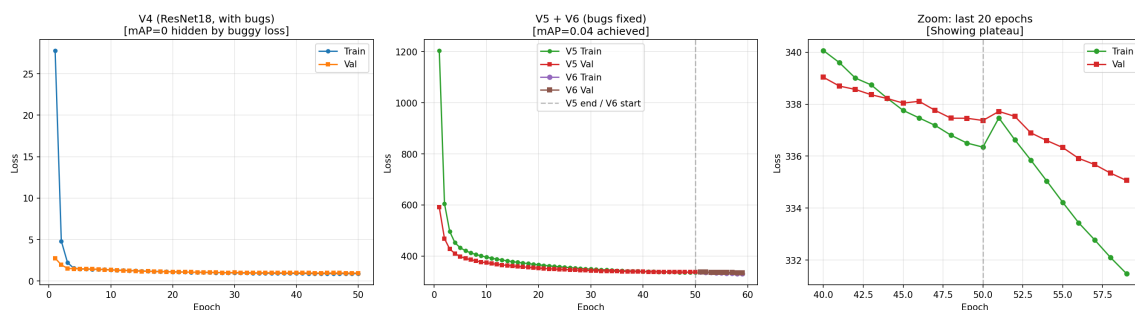


Figura 3: Pérdida de entrenamiento y validación a lo largo de las ejecuciones. Izquierda: V4 (con errores) en meseta cerca de 1,0. Centro: V5+V6 (con correcciones) en meseta cerca de 337. Derecha: *zoom* sobre las últimas diez épocas de V5+V6, que muestra la mejora lenta que la escala absoluta del panel central oculta. El salto de magnitud entre V4 y V5 es por sí mismo la firma de la corrección discutida en la Sección 4.6.

Un lector que llegue a V5 en el registro de entrenamiento sin el contexto de la Sección 4.6 podría razonablemente confundir el aumento de la pérdida con una regresión. Subrayamos que la comparación entre V4 y V5 es una comparación entre dos pérdidas que calculan cantidades fundamentalmente distintas, no entre una versión bien entrenada y otra mal entrenada de la misma pérdida.

5. Resultados

Los resultados de esta sección se calculan sobre el conjunto de prueba reservado de COCO (5% del dataset consolidado, 6 165 imágenes, 42 795 cajas delimitadoras de referencia tras la eliminación de las anotaciones de tipo *crowd*; véase la discusión sobre el descarte por la regla de un objeto por celda en la Sección 4.1). Todas las cifras corresponden al *checkpoint* de V6 (`v6_run/best_v6.pth`) introducido en la Sección 4.8. La inferencia se realiza con *non-maximum suppression* por clase con umbral de IoU de 0,5 y se ejecuta para dos umbrales de probabilidad de predicción, 0,1 (permisivo) y 0,3 (más estricto), con el fin de caracterizar el compromiso precisión/*recall* en lugar de fijar un único punto de operación.

5.1. Cifra titular

La Tabla 2 reporta el mean Average Precision a IoU 0,5 (mAP@0.5) sobre el conjunto de prueba de COCO para ambos umbrales de probabilidad. La cifra titular del proyecto, usada en el resumen y en las conclusiones, es $\text{mAP}@0,5 = 0,0415$ con umbral 0,1.

Tabla 2: Métricas de detección finales del *checkpoint* de V6 sobre el conjunto de prueba de COCO. Promediadas sobre las 80 clases de COCO, todas con al menos una instancia de referencia en el conjunto de prueba.

Umbral de probabilidad	mAP@0.5	Clases con AP > 0
0.1	0.0415	60 / 80
0.3	0.0381	58 / 80

No reclamamos competitividad con los detectores actuales: el propósito de la cifra es acotar el rendimiento absoluto de una implementación fiel al artículo de YOLOv1, redirigida a COCO con red troncal ResNet18 y entrenada durante ~ 60 épocas sobre una sola GPU. El resto de esta sección se dedica a caracterizar *dónde* gasta el modelo, y dónde no, ese presupuesto de detección.

5.2. Desglose por clase

De las 80 clases de COCO evaluadas con umbral 0,1, la distribución del AP está fuertemente sesgada: el decil superior (ocho clases) acapara el grueso del valor de mAP, mientras que una larga cola de veinte clases marca exactamente cero. La tabla ordenada completa para ambos umbrales se proporciona en el Apéndice A; aquí discutimos la estructura de la distribución.

Clases de cabeza. La Tabla 3 lista las diez clases con mayor AP con umbral de probabilidad 0,1. El patrón es consistente: las clases con siluetas grandes y estructuralmente distintivas (rayas de cebra, cuello de jirafa, fuselajes alargados de tren y avión) y las clases que tienden a fotografiarse como sujeto principal de la imagen (gato, oso, elefante, autobús) dominan la cabeza de la distribución.

Tabla 3: Top-10 de clases por AP@0.5 (V6, umbral 0,1). La columna “GT en test” es el número de instancias de referencia de la clase en el conjunto de prueba.

Clase	AP@0.5	GT en test	Idx clase
zebra	0.3593	236	22
train	0.3265	212	6
giraffe	0.3107	225	23
cat	0.2575	255	15
airplane	0.1857	238	4
pizza	0.1795	257	53
bear	0.1698	67	21
elephant	0.1634	223	20
bus	0.1595	246	5
toilet	0.1435	237	61

Clases con $AP = 0$. Con umbral 0,1, veinte clases de COCO registran cero detecciones que superen el criterio de IoU del mAP. La inspección de este conjunto revela cuatro modos de fallo distintos, cada uno consistente con las debilidades documentadas de YOLOv1 (Redmon et al., 2016) y amplificado en nuestro entorno particular.

- *Objetos menores que una celda de la rejilla.* Con $S = 7$ y entrada de 448×448 píxeles, cada celda cubre un parche de 64×64 . Cepillos de dientes, secadores, tijeras y mandos a distancia ocupan sustancialmente menos que esta huella en una fotografía típica. La restricción de YOLOv1 de un objeto por celda implica que la cabeza de detección no tiene forma de localizar múltiples objetos sub-celda, y el *target* de regresión para el único objeto que sí intenta localizar queda dominado por el ruido de cuantización.
- *Cubertería fina o alargada en escenas concurridas.* Tenedor, cuchillo, cuchara y *hot dog* aparecen con más frecuencia en escenas de mesa de comedor o de bandeja de comida donde muchos de ellos se agrupan en una sola celda. Se aplica la misma restricción: solo uno puede ser “responsable” por celda.
- *Clases raras.* Secador de pelo (5 instancias de referencia en el conjunto de prueba), tostadora (9), parquímetro (59), tijeras (78), cepillo de dientes (80) y señal de stop (76) tienen tan pocas instancias de entrenamiento que el modelo ha tenido oportunidad limitada de aprender una plantilla robusta para ellas. La excepción entre las clases de cabeza, *bear* (67 GT en test), muestra que la rareza por sí sola no es suficiente para llevar el AP a cero; la rareza *combinada con* una silueta pequeña o poco distintiva sí lo es.
- *Objetos deportivos en primeros planos humanos.* Pelota, frisbee, bate de béisbol, guante de béisbol y *snowboard* aparecen típicamente en escenas dominadas por figuras humanas con el objeto parcialmente ocluido o cerca de la periferia. La pérdida de clase para la celda responsable compite entonces directamente con el gradiente mucho más fuerte y consistente procedente de la instancia “persona” colocalizada.

El desacoplamiento entre frecuencia y AP. Una expectativa ingenua sería que el AP creciera monotónicamente con el número de instancias de referencia vistas durante el entrenamiento. Los datos lo refutan claramente. La clase “person” es la más común de COCO con gran margen (8 779 instancias de referencia en el conjunto de prueba, aproximadamente el 17 % del total de anotaciones) y alcanza únicamente $AP = 0,035$. La clase “car” (1 382 GT) alcanza 0,005. “Chair” (1 336 GT) alcanza 0,002. Por el contrario, zebra y giraffe, con menos de 240 instancias cada una, están en la cabeza. El factor discriminante no es la frecuencia sino una combinación de tamaño, distintividad y contexto de la imagen.

Efecto del umbral de probabilidad. Pasar de umbral 0,1 a 0,3 hace bajar el mAP aproximadamente un 8 % (de 0,0415 a 0,0381). La mayor parte de la pérdida absoluta procede de clases cuya salida del modelo es incierta incluso cuando es correcta: *zebra* pierde 2,2 puntos porcentuales de AP y *giraffe* 1,2, mientras que *train* prácticamente no se ve afectado (−0,2 puntos). El conjunto de clases con AP nulo crece en dos (*mouse* y *cell phone*, que pasan de

$\sim 10^{-4}$ a 0). El desplazamiento es consistente con una calibración en la cual los verdaderos positivos, en este punto del entrenamiento, se concentran cerca de los valores de confianza más bajos en lugar de los más altos.

5.3. Análisis cualitativo

Ilustramos el comportamiento del modelo con ejemplos extraídos del conjunto de prueba. La selección se divide en tres categorías subjetivas: *good* (cada objeto de referencia se detecta con caja bien localizada, clase correcta y pocos falsos positivos), *medium* (el sujeto principal se reconoce pero la localización es imprecisa, o se omite uno de varios objetos) y *bad* (la predicción está esencialmente descorrelacionada con la referencia, o la única caja detectada es un falso positivo de clase distinta).



Figura 4: Detecciones *good* (*checkpoint* de V6, umbral de probabilidad 0,3). El factor común a través del conjunto bueno es un sujeto grande, situado en posición central y perteneciente a una de las clases de cabeza de la distribución analizada en la Sección 5.2.



Figura 5: Detecciones de calidad media. El detector identifica el objeto dominante de la escena pero con localización imperfecta, u omite objetos secundarios cuyo centro cae en celdas ya reclamadas por otra instancia.

La partición cualitativa corrobora el análisis por clase: cada ejemplo del conjunto *good*

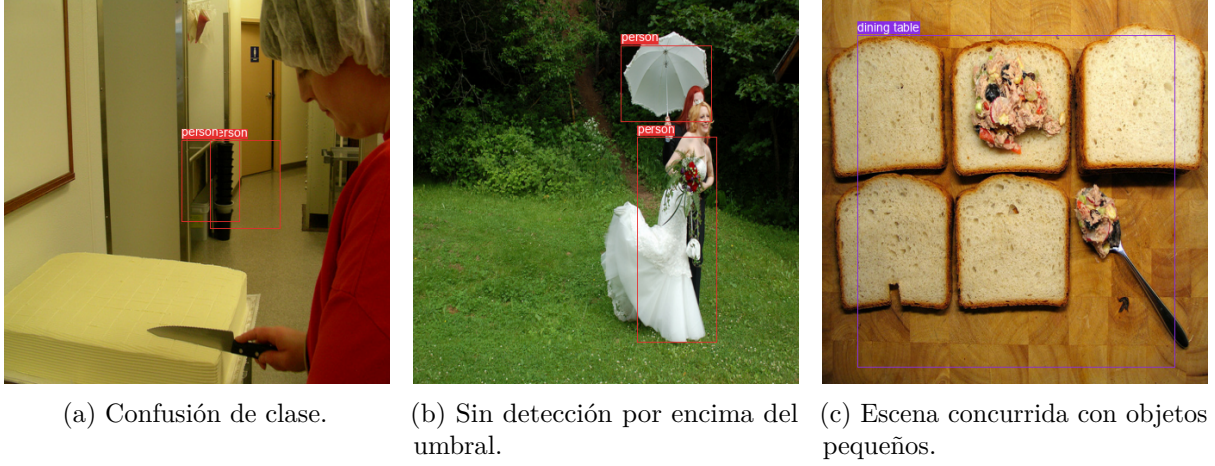


Figura 6: Casos de fallo. O bien la predicción pertenece a una clase distinta a la de referencia (izquierda), o ninguna predicción sobrevive al filtro de probabilidad (centro), o la escena contiene objetos pequeños por debajo de la resolución efectiva de la rejilla $S = 7$ (derecha).

presenta un único sujeto dominante de una clase de cabeza, el conjunto medio falla en objetos secundarios o en localización imprecisa, y el conjunto malo está dominado por casos de objetos pequeños y escenas concurridas. No curamos los ejemplos malos para evitar las clases de cabeza; los fallos malos se producen simplemente fuera de la cabeza.

5.4. Latencia de inferencia

Aunque la motivación original de YOLOv1 era la inferencia en tiempo real, la latencia no es el eje central de este trabajo, porque la investigación de errores en tiempo de entrenamiento de la Sección 4.6 domina la contribución. Por completitud, reportamos la latencia de la pasada hacia adelante del *checkpoint* de V6 medida con `scripts/realtime_detect.py` en su modo `--benchmark`, que ejecuta el modelo sobre entradas sintéticas de tamaño de *batch* uno tras un calentamiento, y registra el tiempo de reloj por llamada con barreras explícitas `torch.cuda.synchronize` antes y después de cada pasada hacia adelante.

Tabla 4: Latencia de inferencia hacia adelante del modelo V6 en una NVIDIA GeForce RTX 5070 Ti Laptop, *batch* 1, entrada 448×448 . Estadísticas sobre 50 iteraciones tras un calentamiento de 20 iteraciones. Memoria pico de GPU asignada: $\sim 1,9$ GB.

Estadístico	Latencia (ms)	Throughput (FPS)
media	5.32	188.1
mediana	5.24	190.9
p_{95}	6.27	159.5
p_{99}	6.76	147.9

El modelo supera con holgura el umbral de tiempo real típicamente asociado al artículo original, que reporta 45 FPS para el modelo YOLO base sobre una Titan X. La diferencia se explica casi por completo por el hardware (una GPU portátil de generación Ada frente a una Titan X de 2016), no por la arquitectura: el número de parámetros de nuestro modelo (248 M) es de hecho mayor que el del artículo, pero está dominado por una sola capa densa cuya evaluación es trivial

sobre hardware moderno. Cuando el mismo script se ejecuta en su modo interactivo de webcam — que añade captura de *frame*, conversión BGR→RGB, NMS, dibujo y ventana gráfica al bucle de inferencia —, el *throughput* es correspondientemente menor y queda limitado sobre todo por el techo de *frame rate* de la webcam.

5.5. Comparación con el artículo original

La Tabla 5 contrasta la configuración y el resultado titular de este trabajo con las cifras reportadas por Redmon et al. (2016). Ambos no son directamente comparables como *benchmarks* — los datasets difieren (COCO vs Pascal VOC), y también las redes troncales y los presupuestos de entrenamiento — pero la comparación es útil para acotar el orden de magnitud de la diferencia y explicitar qué factores contribuyentes son deliberados (dataset, red troncal) y cuáles pragmáticos (duración del entrenamiento, presupuesto de aumentado).

Tabla 5: Comparación lado a lado con el artículo original de YOLOv1.

Aspecto	Redmon et al. (2016)	Este trabajo (V6)
Dataset	VOC 2007 + 2012 (20 clases)	COCO 2017 (80 clases)
Red troncal	Darknet24 (propia)	ResNet18 (preentrenada)
Pretrain de la troncal	Manual sobre ImageNet	torchvision IMAGENET1K_V1
Tasa pico	10^{-2} (calendario manual)	10^{-4}
Planificador	Manual ($10^{-3} \rightarrow 10^{-2} \rightarrow 10^{-3} \rightarrow 10^{-4}$)	Calentamiento lineal + coseno
Épocas	135	60 (V5 + V6 combinadas)
Tamaño de <i>batch</i>	64	32
Resolución de entrada	448×448	448×448
Rejilla (<i>S</i>)	7	7
Cajas por celda (<i>B</i>)	2	2
Canales por celda	$20 + 5 \cdot 2 = 30$	$80 + 5 \cdot 2 = 90$
Aumentado	<i>Crops</i> , exposición, saturación, escalado	Solo <i>resize</i>
λ_{coord}	5	5
λ_{noobj}	0.5	0.5
Hardware	NVIDIA Titan X (2016)	NVIDIA H200 NVL (2024)
Tiempo de entrenamiento	~ 1 semana	~ 2 horas
mAP@0.5	63.4 (VOC 2007 test)	4.15 (COCO test, umbral 0.1)

La diferencia de aproximadamente $\times 15$ en el mAP titular está dominada por cuatro factores que discutimos en detalle en la Discusión: (i) COCO es un *benchmark* sustancialmente más difícil que VOC, tanto por cantidad de clases como por distribución de tamaño de objetos; (ii) el artículo original entrena durante más del doble de tiempo; (iii) ResNet18 es aproximadamente $3\times$ más pequeña en parámetros que Darknet24 en la red troncal (aunque nuestro modelo total es mayor por la cabeza FC); y (iv) el pipeline de aumentado del artículo es considerablemente más rico que nuestra transformación de solo *resize*.

Deliberadamente no interpretamos la diferencia residual como evidencia de un problema de implementación posterior a V5/V6: la investigación de errores en la Sección 4.6 caracterizó los errores de implementación de forma exhaustiva, las verificaciones de *round-trip* y sintéticas certificaron que la pérdida y el decodificador de coordenadas son ahora correctos, y el margen restante es la consecuencia esperable de los cuatro factores anteriores. La sección de Discusión esboza qué sería necesario para cerrar parte de esta diferencia dentro de la misma familia

arquitectónica.

6. Discusión

Esta sección interpreta las cifras reportadas en la Sección 5 a la luz de la investigación de errores de la Sección 4.6 y de las elecciones arquitectónicas de la Sección 3.2, distingue los modos de fallo inherentes a la formulación de YOLOv1 de los específicos de nuestro régimen de entrenamiento, enumera las lecciones que generalizan más allá de este proyecto y esboza las modificaciones que más directamente elevarían el rendimiento absoluto dentro de la misma familia de detectores.

6.1. Por qué el mAP absoluto es bajo

La cifra titular de $\text{mAP}@0,5 = 0,0415$ sobre el conjunto de prueba de COCO es aproximadamente quince veces inferior al 63,4 que Redmon et al. (2016) reportan sobre VOC 2007. Cuatro factores, cada uno independiente de la corrección de la implementación, contribuyen de forma aproximadamente multiplicativa a esta diferencia.

COCO es un benchmark sustancialmente más difícil que VOC. El número de clases sube de 20 a 80, lo que por sí mismo diluye el promedio entre más categorías propensas a fallo. Más importante aún, la distribución del tamaño de los objetos se desplaza: COCO contiene una larga cola de objetos pequeños (una fracción sustancial de las anotaciones son menores que la huella de 64×64 de una sola celda en nuestra rejilla con $S = 7$; véase la Sección 5.2), y muchas de sus escenas están agrupadas en primer plano de un modo que VOC no lo está (varias piezas de cubertería sobre mesas de comedor, varias personas en calles, varios objetos deportivos en fotografías de primer plano). La restricción de un objeto por celda de YOLOv1 se activa con mucha más frecuencia sobre COCO que sobre VOC.

El entrenamiento dura aproximadamente la mitad que el del artículo. V6 entrena durante 60 épocas en total; Redmon et al. (2016) entrenan durante 135. El planificador coseno en nuestra ejecución alcanza su valle hacia la época ~ 45 , por lo que épocas adicionales a esa tasa de aprendizaje no es probable que extraigan mucho más de la configuración actual; un nuevo calendario con tasa pico inferior, en cambio, sí podría.

La red troncal es más pequeña que la del artículo. ResNet18 acapara $\sim 11,2$ M parámetros en el tronco convolucional; Darknet24 (Redmon et al., 2016) acapara ~ 39 M. ResNet18 tiene también un campo receptivo menos profundo en la resolución espacial relevante, lo que limita la información contextual disponible para una predicción de celda individual. Elegimos ResNet18 por tratabilidad (un *checkpoint* preentrenado distribuido por torchvision que encaja en un presupuesto de 50 épocas sobre una H200), no por ser óptima para la tarea. La compensación pagada en mAP titular es la obvia.

El aumentado de datos es mínimo. Redmon et al. (2016) aplican escalado aleatorio, traslación, jitter de exposición y de saturación, además del *flip* horizontal estándar. Nuestro pipeline solo aplica *resize*. Los benchmarks de detección son particularmente sensibles al aumentado de escala porque cada muestra de entrenamiento enseña a la red sobre un único tamaño absoluto de objeto, y sin aumentado la distribución de tamaños entrenados es exactamente la distribución del conjunto de entrenamiento. El aumentado por mosaicos, introducido por variantes posteriores de YOLO, también ayudaría al densificar el número de objetos por imagen de entrenamiento.

El efecto combinado de estos cuatro factores es consistente con una penalización en mAP del orden de un factor diez, que es lo que observamos. De manera crucial, ninguno de los cuatro es un error de implementación: son elecciones deliberadas o pragmáticas realizadas dentro del alcance del proyecto. Los errores de la Sección 4.6 están en un eje totalmente distinto, y se ha verificado que están ausentes en V5–V6 mediante los chequeos de *round-trip* discutidos en esa sección y reforzados por los tests añadidos junto con las correcciones.

6.2. Por qué fallan ciertas clases

El desglose por clase de la Sección 5.2 identificó cuatro modos de fallo recurrentes entre las veinte clases de COCO con $AP = 0$: objetos sub-celda, cubertería agrupada, clases raras y objetos deportivos en primeros planos dominados por humanos. Cada uno es una interacción entre una elección arquitectónica y una propiedad del dataset.

Objetos sub-celda y la rejilla con $S = 7$. Cada celda de la rejilla cubre una región de 64×64 de la entrada 448×448 . Un objeto cuya caja delimitadora sea menor que esta en ambas dimensiones no puede localizarse con precisión por la cabeza de regresión: el *target* de regresión del centro (x, y) se cuantiza dentro de la celda, y los *targets* $\sqrt{w}, \sqrt{h}$ se atraen hacia valores cuyo rango dinámico la red tiene oportunidad limitada de aprender. Cepillos de dientes, secadores, tijeras y mandos a distancia caen en este régimen en una fracción sustancial de sus apariciones en COCO.

Cubertería agrupada y la regla de un objeto por celda. Cuando varios objetos de la misma clase comparten una celda en la referencia, solo uno se retiene como *target* de regresión; el resto se descarta silenciosamente. Las escenas de tenedor/cuchillo/ cuchara son el caso canónico, pero el mismo mecanismo se aplica a varias aves en un cable, varias botellas en una estantería o varios coches en un aparcamiento. En tiempo de evaluación, las instancias omitidas cuentan como falsos negativos, deprimiendo el *recall* y por tanto el AP.

Clases raras. Secador de pelo (5 instancias de referencia en el conjunto de prueba), tostadora (9), parquímetro (59) y clases similares de baja frecuencia no han visto suficientes ejemplos de entrenamiento como para aprender una plantilla robusta. La excepción es *bear* (67 instancias en test, $AP = 0,17$): la rareza por sí sola no descalifica, siempre que la silueta sea lo bastante distintiva como para aprenderse a partir de unos pocos cientos de ejemplos de entrenamiento.

Competencia en primer plano. Los objetos deportivos (pelota, frisbee, bate y guante de béisbol, snowboard) aparecen con frecuencia en escenas cuyo objeto dominante es una persona. La pérdida de clase para la celda responsable compite entonces directamente con el gradiente mucho más fuerte y consistente procedente de la instancia “persona” colocalizada. El resultado es una celda que aprende a clasificar la escena como “persona” con independencia de qué clase posea realmente el centro de la celda.

Ninguno de estos modos de fallo es exclusivo de nuestra implementación; Redmon et al. (2016) discute varios de ellos como limitaciones conocidas de YOLOv1, en particular la incapacidad para manejar agrupaciones de objetos pequeños. Lo que es específico de nuestro entorno es el *grado* en que perjudican a la métrica: COCO activa los cuatro modos de fallo con más frecuencia que VOC, y nuestra red troncal más pequeña deja menos capacidad libre para superarlos.

6.3. Lecciones que generalizan más allá de este proyecto

La investigación de errores en la Sección 4.6 expone un puñado de modos de fallo cuyos detalles son específicos de YOLO pero cuya forma generaliza a cualquier proyecto de aprendizaje profundo que involucre salidas estructuradas. Las resumimos aquí.

Un test que pasa no es evidencia de corrección. Los tests unitarios previos a la corrección para YoloLoss, `convert_cellboxes` y el dataset todos pasaban; el resultado titular fue, sin embargo, cero. Los tests cubrían formas de tensor, *dtype*, rangos de valor y chequeos triviales de sanidad (“la pérdida corre sin fallar”, “la salida tiene la dimensión esperada”) pero no codificaban ningún invariante cuantitativo de lo que la función se supone que calcula. Un test de la forma “codifica una caja conocida, decodifícala, asegura que la caja recuperada coincide” es estructuralmente distinto de uno de la forma “la salida tiene la forma correcta”, y solo el primero habría detectado el Error 2. La misma laguna existe en muchas bases de código de producción y merece auditarse sistemáticamente, no solo cuando un resultado resulta misterioso.

Los *round-trips* de coordenadas merecen chequeos explícitos. Los pipelines de detección mueven cajas delimitadoras a través de tres sistemas de coordenadas distintos (píxeles absolutos, imagen-normalizado, celda-relativo para x, y) y un paso de cuantización (asignación a una celda de la rejilla). Cualquiera de estos pasos puede distorsionar silenciosamente la salida; el efecto acumulado es difícil de depurar a partir solo de las predicciones del modelo. Un *round-trip* explícito — codificar una caja conocida con el código del dataset, decodificarla con el código de post-procesamiento y aserción de igualdad — habría detectado el Error 2 en una sola línea. Añadimos un test de este estilo junto con la corrección.

Las operaciones *in-place* sobre tensores pueden romper autograd. El patrón `in-place box_predictions[..., 2:4] = sign(.) * sqrt(.)` que distribuía una versión anterior de la pérdida era sintácticamente válido, no levantaba excepción, devolvía el valor correcto de la pérdida y rompía silenciosamente la pasada hacia atrás por el camino w, h . PyTorch acepta generalmente este patrón; solo objeta cuando el tensor resultado es también necesario para una operación posterior en la misma pasada hacia adelante. Su detección es esencialmente imposible

sin (i) un test unitario sobre el flujo de gradiente por el camino ofensor, o (ii) el hábito deliberado de expresar transformaciones por celda con `torch.cat` en lugar de asignación indizada.

Una pérdida convergente no es evidencia de un modelo que aprende. La observación más contraintuitiva de la investigación de errores es que V1–V4 producían pérdidas de entrenamiento y validación que descendían monotónicamente a través de las épocas, se estabilizaban en valores de aspecto sensato (alrededor de 1,0 para V4) y daban una fuerte impresión de que el modelo estaba aprendiendo. No lo estaba. La pérdida que minimizaban era una cantidad aritméticamente válida que la red sí podía reducir, pero no era la pérdida YOLO; era una proyección de la pérdida YOLO sobre un subconjunto de los canales de salida que no incluía los huecos de coordenadas de caja. Una generalización útil: cuando un modelo que debería detectar cosas no detecta nada, la pregunta “¿es la pérdida la pérdida?” debería tener prioridad sobre la pregunta “¿están afinados los hiperparámetros?”.

6.4. Trabajo futuro

Las rutas más directas para un mAP absoluto más alto, ordenadas por impacto esperado dentro de las restricciones de la familia arquitectónica de YOLOv1, son:

1. *Red troncal mayor con presupuesto comparable.* Sustituir ResNet18 por ResNet50 triplica aproximadamente el número de parámetros del tronco y dobla aproximadamente la profundidad convolucional. Sobre clasificación de ImageNet esto vale varios puntos porcentuales de *top-1 accuracy*; la ganancia correspondiente en detección sobre COCO debería superar a la de un único barrido de hiperparámetros con la red troncal actual.
2. *Rejilla más fina ($S = 14$).* Una rejilla de 14×14 a la entrada actual de 448×448 reduciría la huella por celda a 32×32 píxeles, por debajo del umbral en el que el modo de fallo de objetos sub-celda de la Sección 6.2 se activa para la mayoría de clases de COCO. Esto exige bien eliminar el *downsampling* del adaptador o duplicar la resolución de entrada a 896×896 (a coste cuadrático en tiempo de inferencia).
3. *Aumentado por mosaicos.* La técnica de mosaicos introducida por las variantes posteriores de YOLO une cuatro imágenes aleatorias del entrenamiento en una única muestra, multiplicando por cuatro el número de objetos por ejemplo de entrenamiento y entrenando indirectamente a la red sobre instancias de objeto en una gama más amplia de contextos. Es barato de implementar y se ha reportado que vale varios puntos de mAP sobre COCO con detectores pequeños.
4. *Cajas ancla.* Redmon and Farhadi (2018) sustituye la asignación de “caja responsable” de YOLOv1 por un pequeño conjunto de priores ancla por celda de la rejilla (obtenidos por k -medias sobre las cajas del conjunto de entrenamiento) y convierte la regresión de la caja de predicción directa de (x, y, w, h) a un *delta* contra cada ancla. Esto desacopla el *target* de regresión de la competencia por celda entre cajas y mejora sustancialmente la localización. Requeriría cambios en el dataset, la pérdida y el post-procesamiento simultáneamente, y es por tanto el ítem más grande de esta lista.

5. *Entrenamiento más largo, tasa más agresiva.* V5–V6 se detienen en el valle del coseno tras 60 épocas totales. Un nuevo calendario de 150 épocas con tasa pico de 5×10^{-4} (intermedia entre el 10^{-3} de V1 y el 10^{-4} de V4) y el umbral de recorte de gradiente existente de 10 probablemente daría mejora adicional, aunque menor que cualquiera de los cuatro ítems anteriores.

Los ítems 1, 3 y 5 son mejoras continuas; el ítem 2 es un cambio arquitectónico pero dentro del marco de YOLOv1; el ítem 4 constituye efectivamente YOLOv2 y sería un sistema sustancialmente distinto.

7. Conclusiones

Este trabajo reproduce el detector YOLOv1 de Redmon et al. (2016) sobre el dataset COCO 2017, utilizando una red troncal ResNet18 (He et al., 2016) preentrenada sobre ImageNet, un adaptador convolucional y la cabeza de detección densa de dos capas del artículo. El pipeline — preparación del dataset, arquitectura del modelo, pérdida de cinco términos, bucle de entrenamiento distribuido con precisión mixta y evaluador de mAP con *non-maximum suppression* por clase — está implementado en PyTorch y es reproducible extremo a extremo a partir de la partición del dataset con semilla fija.

La cifra titular es $\text{mAP}@0,5 = 0,0415$ sobre el conjunto de prueba reservado de COCO (*checkpoint* V6, umbral de probabilidad 0,1), con una latencia de pasada hacia adelante de $\sim 5,3$ ms por imagen a 448×448 sobre una única GPU portátil de generación Ada, holgadamente dentro del régimen de tiempo real que perseguía el artículo original. Tanto el mAP absoluto como la diferencia con el 63,4 del artículo sobre VOC son atribuibles a elecciones deliberadas hechas dentro del presupuesto del proyecto y no a defectos de implementación: el dataset es aproximadamente cuatro veces más difícil (80 clases frente a 20, con una larga cola de objetos pequeños y agrupados), el calendario de entrenamiento es la mitad de duración, la red troncal es sustancialmente más pequeña, y el pipeline de aumentado es mínimo.

El hallazgo central del proyecto es metodológico antes que numérico. Las cuatro primeras ejecuciones de entrenamiento (V1 a V4), que cubren dos redes troncales distintas y un barrido deliberadamente amplio de tasas de aprendizaje y umbrales de recorte de gradiente, produjeron todas exactamente $\text{mAP}@0,5 = 0$ sobre el conjunto de prueba a pesar de que las pérdidas de entrenamiento y validación descendían monotónicamente y se estabilizaban en valores que un observador interpretaría naturalmente como evidencia de un modelo que aprende. La causa no fue una mala elección de hiperparámetros ni una limitación fundamental de la formulación de YOLO, sino dos errores de implementación independientes:

- Una función de pérdida que indizaba el tensor de predicción por celda con posiciones apropiadas para la disposición de Pascal VOC ($C = 20$) mientras recibía *targets* de COCO ($C = 80$). De 1062 celdas con objeto de referencia en una muestra de 200 imágenes de entrenamiento, la máscara errónea “*exists_box*” estaba activada únicamente para tres de ellas (los elefantes). El modelo era ciego al gradiente sobre los huecos de caja delimitadora que se suponía que debía predecir, y la señal de localización completa se evaporó.

- Una función de post-procesamiento que dividía las anchuras y alturas predichas por $S = 7$ antes de devolverlas, aun cuando el dataset y la pérdida habían acordado expresar w, h directamente en unidades relativas a la imagen. Incluso un modelo entrenado a la perfección habría producido cajas siete veces más pequeñas que la referencia, con una IoU máxima alcanzable de $\sim 0,02$, muy por debajo del umbral de 0,5 del mAP@0,5.

Cada error, por sí solo, basta para producir mAP@0,5 = 0 en tiempo de evaluación. Ambos eran invisibles para la batería de tests preexistente, que comprobaba formas de tensor y rangos de valor pero no invariantes cuantitativos de coordenadas o de valor. Ambos se detectaron dentro de una sola pasada de auditoría una vez que la pregunta cambió de “¿por qué están mal los hiperparámetros?” a “¿es la pérdida la pérdida?”.

La conclusión pedagógica, más amplia que los errores concretos, es que un pipeline de detección contiene muchos modos de fallo silenciosos cuyos síntomas individuales son demasiado sutiles para diagnosticarse solo desde la inspección de las curvas de entrenamiento. Una curva de pérdida convergente es consistente tanto con un modelo que aprende lo que queremos que aprenda como con un modelo que aprende una cantidad aritméticamente válida que no es lo que queremos. La distinción puede resolverse barata con tests de *round-trip* sobre cada transformación de coordenadas y con tests de pérdida que ejerciten el número de clases de producción y no únicamente aquel para el que se escribió originalmente el test. Hemos añadido ambos tipos de test junto con las correcciones; el proyecto habría llegado a mAP@0,5 > 0 dentro de la primera ejecución de entrenamiento si hubieran existido desde el inicio.

Si el detector resultante habría sido competitivo con los detectores modernos de una sola etapa es una pregunta que la arquitectura de YOLOv1 nunca fue diseñada para responder. Las mejoras que cierran gran parte de la diferencia residual con sus descendientes — cajas ancla, pirámides de características multi-escala, aumentado por mosaicos, cabezas totalmente convolucionales — son a estas alturas el repertorio estándar de la familia YOLO desde [Redmon and Farhadi \(2018\)](#) en adelante, y constituyen la continuación natural de este trabajo.

Referencias

- Everingham, M., Van Gool, L., Williams, C. K. I., Winn, J., and Zisserman, A. (2010). The pascal visual object classes (voc) challenge. *International Journal of Computer Vision*, 88(2):303–338.
- Girshick, R. (2015). Fast r-cnn. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*.
- Girshick, R., Donahue, J., Darrell, T., and Malik, J. (2014). Rich feature hierarchies for accurate object detection and semantic segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
- He, K., Zhang, X., Ren, S., and Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778.

- Ioffe, S. and Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International Conference on Machine Learning (ICML)*.
- Krizhevsky, A., Sutskever, I., and Hinton, G. E. (2012). Imagenet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 25.
- Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., and Zitnick, C. L. (2014). Microsoft coco: Common objects in context. In *European Conference on Computer Vision (ECCV)*.
- Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C.-Y., and Berg, A. C. (2016). Ssd: Single shot multibox detector. In *European Conference on Computer Vision (ECCV)*.
- Loshchilov, I. and Hutter, F. (2017). Sgdr: Stochastic gradient descent with warm restarts. *International Conference on Learning Representations (ICLR)*.
- Micikevicius, P., Narang, S., Alben, J., Diamos, G., Elsen, E., Garcia, D., Ginsburg, B., Houston, M., Kuchaiev, O., Venkatesh, G., and Wu, H. (2018). Mixed precision training. *International Conference on Learning Representations (ICLR)*.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., et al. (2019). Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Persson, A. (2020). Yolov1 from scratch (open-source implementation). https://github.com/aladdinpersson/Machine-Learning-Collection/tree/master/ML/Pytorch/object_detection/YOLO. Accessed: May 2026.
- Redmon, J., Divvala, S., Girshick, R., and Farhadi, A. (2016). You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 779–788.
- Redmon, J. and Farhadi, A. (2018). Yolov3: An incremental improvement. *arXiv preprint arXiv:1804.02767*.
- Ren, S., He, K., Girshick, R., and Sun, J. (2015). Faster r-cnn: Towards real-time object detection with region proposal networks. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., Huang, Z., Karpathy, A., Khosla, A., Bernstein, M., Berg, A. C., and Fei-Fei, L. (2015). Imagenet large scale visual recognition challenge. *International Journal of Computer Vision*, 115(3):211–252.
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., and Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. In *Journal of Machine Learning Research*, volume 15, pages 1929–1958.

A. Average Precision por clase (V6, tabla completa)

La Tabla 6 lista el Average Precision a IoU 0,5 para cada una de las 80 clases de COCO, evaluado sobre el conjunto de prueba reservado (6 165 imágenes, 42 795 cajas delimitadoras de referencia tras la eliminación de las anotaciones de tipo *crowd*) con el *checkpoint* de V6. Se reportan dos umbrales de probabilidad, 0,1 (la configuración permisiva de la que se promedia el mAP titular de 0,0415) y 0,3 (la configuración más estricta que arroja 0,0381). La columna “GT” da el número de instancias de referencia de la clase en el conjunto de prueba, lo que aporta contexto tanto al modo de fallo de clases raras discutido en la Sección 6.2 como al desacoplamiento entre frecuencia y AP discutido en la Sección 5.2. La fuente completa es `artefacts/ap_per_class/per_class_ap_best_v6.pth_{0.1,0.3}.csv`.

Tabla 6: AP@0.5 por clase sobre el conjunto de prueba de COCO (*checkpoint* V6). Filas ordenadas por índice de clase. En **negrita** se marcan las clases dentro del top-10 con umbral 0,1; una daga ([†]) marca las clases con AP = 0 con umbral 0,1.

Idx	Clase	AP@0.5 ($p \geq 0,1$)	AP@0.5 ($p \geq 0,3$)	GT
0	person	0.0350	0.0337	8 779
1	bicycle	0.0014	0.0002	300
2	car	0.0052	0.0051	1 382
3	motorcycle	0.0891	0.0855	303
4	airplane	0.1857	0.1799	238
5	bus	0.1595	0.1595	246
6	train	0.3265	0.3248	212
7	truck	0.0228	0.0228	357
8	boat	0.0002	0.0001	370
9	traffic light	0.0003	0.0002	451
10	fire hydrant	0.0435	0.0353	85
11	stop sign	0.0502	0.0433	76
12	parking meter [†]	0.0000	0.0000	59
13	bench	0.0110	0.0081	362
14	bird	0.0540	0.0529	384
15	cat	0.2575	0.2287	255
16	dog	0.0826	0.0745	310
17	horse	0.0562	0.0497	244
18	sheep	0.0373	0.0341	473
19	cow	0.0397	0.0333	261
20	elephant	0.1634	0.1466	223
21	bear	0.1698	0.1403	67
22	zebra	0.3593	0.3372	236
23	giraffe	0.3107	0.2988	225
24	backpack [†]	0.0000	0.0000	285

continúa en la página siguiente

Tabla 6 continuación de la página anterior.

Idx	Clase	AP@0.5 ($p \geq 0,1$)	AP@0.5 ($p \geq 0,3$)	GT
25	umbrella	0.0012	0.0009	384
26	handbag [†]	0.0000	0.0000	272
27	tie	0.0087	0.0047	269
28	suitcase	0.0001	0.0001	187
29	frisbee [†]	0.0000	0.0000	117
30	skis	0.0185	0.0180	250
31	snowboard [†]	0.0000	0.0000	111
32	sports ball [†]	0.0000	0.0000	231
33	kite	0.0017	0.0014	355
34	baseball bat [†]	0.0000	0.0000	145
35	baseball glove [†]	0.0000	0.0000	108
36	skateboard	0.0064	0.0053	241
37	surfboard	0.0165	0.0152	215
38	tennis racket	0.0015	0.0015	168
39	bottle	0.0001	0.0001	860
40	wine glass	0.0002	0.0002	247
41	cup	0.0035	0.0027	665
42	fork [†]	0.0000	0.0000	262
43	knife [†]	0.0000	0.0000	251
44	spoon [†]	0.0000	0.0000	190
45	bowl	0.0015	0.0015	557
46	banana	0.0053	0.0043	399
47	apple	0.0028	0.0017	155
48	sandwich	0.0220	0.0100	174
49	orange	0.0016	0.0016	185
50	broccoli	0.0085	0.0062	375
51	carrot	0.0001	0.0001	289
52	hot dog [†]	0.0000	0.0000	128
53	pizza	0.1795	0.1651	257
54	donut	0.0120	0.0082	214
55	cake	0.0007	0.0002	267
56	chair	0.0022	0.0019	1 336
57	couch	0.0029	0.0026	275
58	potted plant [†]	0.0000	0.0000	368
59	bed	0.1215	0.1017	215
60	dining table	0.0376	0.0372	585
61	toilet	0.1435	0.1333	237
62	tv	0.0084	0.0063	265
63	laptop	0.0170	0.0137	219

continúa en la página siguiente

Tabla 6 continuación de la página anterior.

Idx	Clase	AP@0.5 ($p \geq 0,1$)	AP@0.5 ($p \geq 0,3$)	GT
64	mouse	0.0002	0.0000	97
65	remote [†]	0.0000	0.0000	242
66	keyboard	0.0173	0.0143	135
67	cell phone	0.0001	0.0000	279
68	microwave [†]	0.0000	0.0000	81
69	oven	0.0098	0.0093	152
70	toaster [†]	0.0000	0.0000	9
71	sink	0.0014	0.0012	288
72	refrigerator	0.0098	0.0063	124
73	book [†]	0.0000	0.0000	667
74	clock	0.0973	0.0958	292
75	vase	0.0553	0.0458	227
76	scissors [†]	0.0000	0.0000	78
77	teddy bear	0.0433	0.0389	200
78	hair drier [†]	0.0000	0.0000	5
79	toothbrush [†]	0.0000	0.0000	80

Estadísticas resumen. Con umbral 0,1, el AP medio sobre las 80 clases (es decir, el mAP@0.5 titular) es 0,0415. Veinte clases alcanzan $AP = 0$ (marcadas con [†]). Diez clases alcanzan $AP > 0,1$; son las que aparecen en negrita en la columna “AP@0.5 ($p \geq 0,1$)” y también las entradas de la Tabla 3. Con umbral 0,3, la media desciende a 0,0381 con veintidós clases de AP nulo: dos clases adicionales caen a cero con el umbral más estricto, en concreto *mouse* (de 2×10^{-4}) y *cell phone* (de 10^{-4}).

B. Función de pérdida: antes y después de la corrección de índices

Este apéndice muestra el cuerpo de `YoloLoss.forward` tal y como estaba en las ejecuciones V1–V4, cuando producía $mAP@0,5 = 0$ sobre COCO, junto con la versión posterior a la corrección utilizada en V5 y V6. El diagnóstico y la verificación del error se discuten en la Sección 4.6.1; el propósito del apéndice es que el *diff* en sí sea auditable como un único bloque.

Versión con error (V1–V4). Los índices literales 20, 21:25, 25, 26:30 y :20 son correctos para la disposición por celda de Pascal VOC ($C=20$, total 30 canales) e incorrectos para COCO ($C=80$, total 90 canales). Apuntan a la región de probabilidad de clase del tensor de COCO en lugar de a los huecos de coordenadas y confianza.

```

1 def forward(self, predictions, target):
2     predictions = predictions.reshape(-1, self.S, self.S, self.C + self.B * 5)
3
4     # IoU entre cajas predichas y de referencia (GT asumida en 21:25).
```

```

5     iou_b1 = iou(predictions[..., 21:25], target[..., 21:25])
6     iou_b2 = iou(predictions[..., 26:30], target[..., 21:25])
7     ious   = torch.stack([iou_b1, iou_b2], dim=0)
8     _, best_box = torch.max(ious, dim=0)
9
10    exists_box = target[..., 20:21]                # mascara "existe
        objeto"
11
12    # Perdida de caja (xy, sqrt(wh))
13    box_pred = exists_box * (best_box              * predictions[..., 26:30]
14                          + (1 - best_box) * predictions[..., 21:25])
15    box_tgt  = exists_box * target[..., 21:25]
16    box_pred[..., 2:4] = torch.sign(box_pred[..., 2:4]) * torch.sqrt(
17        torch.abs(box_pred[..., 2:4] + 1e-6))
18    box_tgt[..., 2:4] = torch.sqrt(box_tgt[..., 2:4] + 1e-6)
19    box_loss = mse(flatten(box_pred), flatten(box_tgt))
20
21    # Perdida de confianza de objeto
22    pred_box = best_box              * predictions[..., 25:26] \
23            + (1 - best_box) * predictions[..., 20:21]
24    object_loss = mse(flatten(exists_box * pred_box),
25                    flatten(exists_box * target[..., 20:21]))
26
27    # Perdida de confianza sin objeto (ambas cajas predichas)
28    no_object_loss = mse(flatten((1 - exists_box) * predictions[..., 20:21]),
29                        flatten((1 - exists_box) * target[..., 20:21]))
30    no_object_loss += mse(flatten((1 - exists_box) * predictions[..., 25:26]),
31                        flatten((1 - exists_box) * target[..., 20:21]))
32
33    # Perdida de clase
34    class_loss = mse(flatten(exists_box * predictions[..., :20]),
35                    flatten(exists_box * target[..., :20]))
36
37    return (self.lambda_coord * box_loss
38          + object_loss
39          + self.lambda_noobj * no_object_loss
40          + class_loss)

```

Listing 4: Cuerpo con error de YoloLoss.forward.

Versión corregida (V5–V6). Cada literal se sustituye por una rebanada o un índice derivado de `self.C`, la pérdida de clase se ensancha de `:20` a `:C`, y la asignación *in-place* de raíz cuadrada sobre `box_pred[..., 2:4]` se sustituye por un `torch.cat` de las rebanadas xy y $\sqrt{wh}$ transformada, que preserva el grafo de autograd a través del camino w, h .

```

1 def forward(self, predictions, target):
2     predictions = predictions.reshape(-1, self.S, self.S, self.C + self.B * 5)
3
4     C          = self.C
5     OBJ_IDX    = C
6     BOX1_SLICE = slice(C + 1, C + 5)
7     CONF2_IDX  = C + 5

```

```

8 BOX2_SLICE = slice(C + 6, C + 10)
9
10 iou_b1 = iou(predictions[..., BOX1_SLICE], target[..., BOX1_SLICE])
11 iou_b2 = iou(predictions[..., BOX2_SLICE], target[..., BOX1_SLICE])
12 ious = torch.stack([iou_b1, iou_b2], dim=0)
13 _, best_box = torch.max(ious, dim=0)
14
15 exists_box = target[..., OBJ_IDX:OBJ_IDX+1]
16
17 # Perdida de caja (xy, sqrt(wh)) --- reconstruir el tensor con torch.cat
18 box_pred = exists_box * (best_box * predictions[..., BOX2_SLICE]
19                          + (1 - best_box) * predictions[..., BOX1_SLICE])
20 box_tgt = exists_box * target[..., BOX1_SLICE]
21
22 xy_pred = box_pred[..., 0:2]
23 wh_pred_sqrt = torch.sign(box_pred[..., 2:4]) * torch.sqrt(
24                 torch.abs(box_pred[..., 2:4]) + 1e-6)
25 box_pred = torch.cat([xy_pred, wh_pred_sqrt], dim=-1)
26
27 xy_tgt = box_tgt[..., 0:2]
28 wh_tgt_sqrt = torch.sqrt(box_tgt[..., 2:4] + 1e-6)
29 box_tgt = torch.cat([xy_tgt, wh_tgt_sqrt], dim=-1)
30
31 box_loss = mse(flatten(box_pred), flatten(box_tgt))
32
33 # Perdida de confianza de objeto
34 pred_box = best_box * predictions[..., CONF2_IDX:CONF2_IDX+1] \
35           + (1 - best_box) * predictions[..., OBJ_IDX:OBJ_IDX+1]
36 object_loss = mse(flatten(exists_box * pred_box),
37                  flatten(exists_box * target[..., OBJ_IDX:OBJ_IDX+1]))
38
39 # Perdida de confianza sin objeto (ambas cajas predichas)
40 no_object_loss = mse(flatten((1 - exists_box) * predictions[..., OBJ_IDX:
41 OBJ_IDX+1])),
42                  flatten((1 - exists_box) * target[..., OBJ_IDX:
43 OBJ_IDX+1]))
44
45 no_object_loss += mse(flatten((1 - exists_box) * predictions[..., CONF2_IDX:
46 :CONF2_IDX+1])),
47                  flatten((1 - exists_box) * target[..., OBJ_IDX:
48 OBJ_IDX+1]))
49
50 # Perdida de clase --- las C clases, no solo las 20 primeras
51 class_loss = mse(flatten(exists_box * predictions[..., :C]),
52                  flatten(exists_box * target[..., :C]))
53
54 return (self.lambda_coord * box_loss
55         + object_loss
56         + self.lambda_noobj * no_object_loss
57         + class_loss)

```

Listing 5: Cuerpo corregido de YoloLoss.forward.

Resumen del *diff*. Los cambios semánticos son:

1. Cinco literales codificados (20, 21:25, 25, 25:26, 26:30) sustituidos por variables de índice derivadas de `self.C`.
2. La rebanada de la pérdida de clase :20 ampliada a : C , para que las ochenta clases de COCO contribuyan, no solo las veinte primeras.
3. La asignación *in-place* `box_pred[..., 2:4] = sign(.) * sqrt(.)` sustituida por `torch.cat([xy, sqrt(wh)], dim=-1)` para preservar la pasada hacia atrás de autograd a través de la rebanada w, h .

La corrección es puramente local a `src/loss.py`; ningún otro fichero del repositorio requirió cambios para que la corrección de la pérdida tuviera efecto. La corrección complementaria a la función de post-procesamiento `convert_cellboxes` (Error 2, Sección 4.6.2) es una edición de una sola línea en `src/utils.py` que elimina el factor espurio $1/S$ de la anchura y la altura de las cajas decodificadas.